جامعة القاهرة الجديدة التكنولوجية
NEW CAIRO TECHNOLOGICAL UNIVERSITY

Faculty of Industry and Energy Technology

Information And Communication Program

Graduation Projects for 2025 – 2026

Final Year Project

ANCS — Autonomous Network Configuration & Orchestration System

Submitted by:

Zyad Ahmed Eid Sadek
Youssef Marei Ali Salem
Youssef Samir Mohamed Ibrahim
Alaa Sabry Mahmoud Mohamed
Ahmed Mahmoud Abdelkarim Ahmed
Mohamed Ehab Sayed
Mohamed Bandary Saber Bandary

Supervised by:

Assoc. Prof. Dr. Mohamed Hassan Saad

This project has been submitted in partial fulfillment of the requirements for the Bachelor of Technology Degree, New Cairo Technological University. (2026)

ACKNOWLEDGMENT

We would like to express our deepest gratitude to our supervisor, **Assoc. Prof. Dr. Mohamed Hassan Saad**, whose unwavering support, both academically and personally, has been instrumental throughout the journey of this project. His guidance, encouragement, and belief in our capabilities pushed us to deliver our best work. Beyond the technical mentorship, his moral support during the challenging phases of development was truly invaluable and will always be remembered.

We also extend our thanks to **New Cairo Technological University** for providing us with the foundational knowledge, resources, and environment that made this project possible.

Finally, we would like to thank our families for their patience and support throughout this academic year.

ABSTRACT

Network infrastructure management remains a critical challenge in modern IT environments. The manual configuration of network devices using command-line interfaces (CLI) is time-consuming, error-prone, and requires specialized expertise in vendor-specific syntax. Studies indicate that approximately 80% of network outages are attributed to human configuration errors, highlighting the urgent need for automated solutions.

This project presents **ANCS (Autonomous Network Configuration & Orchestration System)**, a desktop application built with Python and PySide6 that automates Cisco IOS device configuration, integrates with the GNS3 network simulation platform, and features an AI-powered Copilot. ANCS addresses the gap between complex network automation frameworks that require programming expertise and simple educational simulators that lack deployment capabilities.

The system provides three core capabilities: (1) **Guided Configuration Wizards** that derive complete network configurations — including VLANs, routing protocols (OSPF, EIGRP, RIP), DHCP, and access control lists — from minimal user input; (2) **Deep GNS3 Integration** via REST API for topology discovery, node management, and live configuration synchronization; and (3) an **Agentic AI Copilot** powered by Google's Gemini large language model with 34 callable tools enabling autonomous network discovery, configuration generation, deployment, auditing, and connectivity tracing.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

ACL	Access Control List
AI	Artificial Intelligence
ANCS	Autonomous Network Configuration & Orchestration System
API	Application Programming Interface
BGP	Border Gateway Protocol
CLI	Command-Line Interface
CSS	Cascading Style Sheets
DHCP	Dynamic Host Configuration Protocol
EIGRP	Enhanced Interior Gateway Routing Protocol
ERD	Entity-Relationship Diagram
GNS3	Graphical Network Simulator 3
GUI	Graphical User Interface
HITL	Human-in-the-Loop
HTML	HyperText Markup Language
HUD	Heads-Up Display
IAC	Interpret As Command (Telnet)
IOS	Internetwork Operating System (Cisco)
JS	JavaScript
LLM	Large Language Model
MVVM	Model-View-ViewModel

NLP	Natural Language Processing
OSPF	Open Shortest Path First
PDF	Portable Document Format
REST	Representational State Transfer
RIP	Routing Information Protocol
SDN	Software-Defined Networking
SNMP	Simple Network Management Protocol
SQL	Structured Query Language
SSH	Secure Shell
STP	Spanning Tree Protocol
SVI	Switched Virtual Interface
SVG	Scalable Vector Graphics
TCP	Transmission Control Protocol
UAT	User Acceptance Testing
UML	Unified Modeling Language
VLAN	Virtual Local Area Network
VLSM	Variable-Length Subnet Masking
VRP	Versatile Routing Platform (Huawei)
WAL	Write-Ahead Logging

LIST OF UTILIZED STANDARDS

Standard	Description	Usage in ANCS
IEEE 802.1Q	VLAN Tagging Standard	VLAN configuration generation and trunk port setup
IEEE 802.1D	Spanning Tree Protocol	STP configuration in switch device profiles
RFC 2131	Dynamic Host Configuration Protocol (DHCP)	DHCP pool configuration in guided wizard
RFC 2328	OSPF Version 2	OSPF routing protocol configuration
RFC 7868	EIGRP Informational	EIGRP routing protocol configuration
RFC 2453	RIP Version 2	RIPv2 routing protocol configuration
RFC 4251	SSH Protocol Architecture	SSH-based device communication via Paramiko
RFC 854	Telnet Protocol Specification	Telnet-based console access with IAC handling
RFC 1918	Private Internet Address Allocation	Subnet calculator and IP scheme generation
ISO/IEC 25010	Software Quality Requirements	System quality attributes (reliability, usability, performance)

CHAPTER ONE

INTRODUCTION

1.1 BACKGROUND

The rapid expansion of computer networks in both enterprise and educational environments has placed enormous demands on network administrators. Modern networks consist of tens, hundreds, or even thousands of interconnected devices — routers, switches, firewalls, and access points — each requiring precise configuration to ensure proper communication, security, and performance. The predominant method of configuring these devices remains the command-line interface (CLI), a text-based environment that requires administrators to manually type vendor-specific commands on each device individually [1].

Cisco Systems, the market leader in networking equipment, uses its proprietary Internetwork Operating System (IOS) across its router and switch product lines. Cisco IOS provides a powerful but complex CLI that demands deep expertise in networking concepts and syntax. A typical network configuration task — such as setting up VLANs, configuring routing protocols, and implementing access control lists — requires the administrator to execute dozens of commands in the correct order, on the correct device, in the correct configuration mode [2].

Studies have consistently shown that human error is the leading cause of network failures. According to Gartner, approximately 80% of network outages are caused by configuration errors [3]. These errors range from simple typos in IP addresses to logical mistakes in routing protocol

configurations that can bring down entire network segments. The consequences are significant: network downtime costs enterprises an average of $5,600 per minute according to industry estimates [4].

In educational settings, the challenge is compounded. Students learning networking concepts through simulation platforms like GNS3 (Graphical Network Simulator 3) must manually configure each virtual device, a process that diverts time and attention from understanding networking principles to mastering CLI syntax. GNS3 provides a powerful simulation environment that runs real Cisco IOS images, but it offers no built-in automation or configuration assistance capabilities [5].

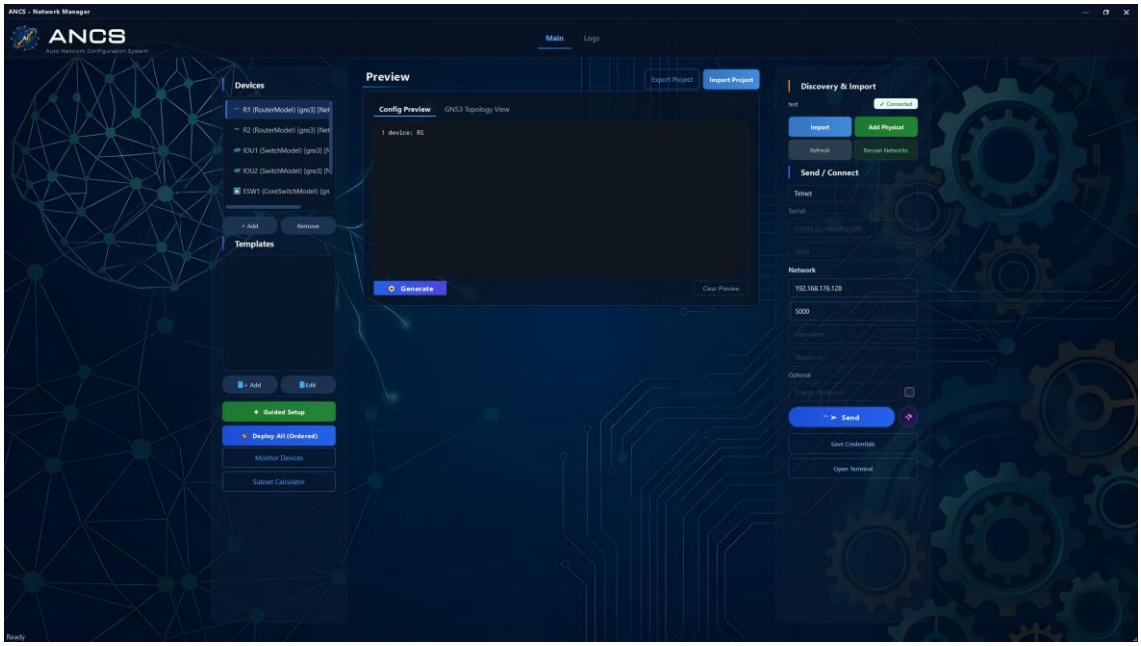

Figure 1.1: ANCS main dashboard interface showing device grid and navigation controls.

The emergence of network automation tools has begun to address these challenges. Frameworks such as Ansible, Netmiko, and NAPALM enable programmatic configuration of network devices. However, these tools require programming expertise in Python or YAML, creating a barrier for network administrators and students who may lack software development

skills. There exists a significant gap between the complexity of professional automation frameworks and the simplicity needed for educational and small-scale network environments [6].

1.2 PROBLEM STATEMENT

Despite advances in network automation, three fundamental problems persist that ANCS aims to address:

Problem 1: Manual Configuration is Error-Prone and Unscalable. Configuring network devices through CLI remains a fundamentally serial, manual process. An administrator must connect to each device, navigate to the correct configuration mode, enter commands one at a time, and verify the results. For a network of 10 devices requiring VLAN, routing, and security configuration, this process can take hours and introduces numerous opportunities for error. Each device requires approximately 30–50 configuration commands, and a single mistyped IP address or missing network statement can cause connectivity failures across the entire topology [3].

Problem 2: Existing Automation Tools Require Programming Expertise. Current network automation solutions such as Ansible require administrators to write YAML playbooks, understand Jinja2 templating, and manage inventory files. Netmiko and NAPALM require Python programming skills. These tools are designed for DevOps professionals and are inaccessible to the majority of network administrators and all networking students who lack programming backgrounds [7]. There is no tool that provides guided, wizard-based configuration generation without requiring the user to write code.

Problem 3: No Integrated Solution Combines Simulation, Automation, and AI Assistance. Network simulation platforms (GNS3, EVE-NG, Packet Tracer) provide topology creation and device emulation but lack automation capabilities. Automation tools (Ansible, Netmiko) provide programmatic configuration but lack visual topology awareness. AI-powered assistants can generate configuration suggestions but lack the ability to directly deploy configurations or interact with live devices. No existing tool integrates all three capabilities — visual topology management, guided configuration generation, and AI-powered autonomous assistance — into a single desktop application [8].

1.3 PROJECT OBJECTIVES

The primary objective of this project is to develop ANCS (Autonomous Network Configuration & Orchestration System), a desktop application that addresses the identified problems through the following specific objectives:

1. **Automate Cisco IOS Configuration Generation:** Develop guided wizards that derive complete, deployment-ready network configurations from minimal user input, supporting VLANs, routing protocols (OSPF, EIGRP, RIP), DHCP, access control lists, and inter-VLAN routing.
2. **Integrate with GNS3 Network Simulation:** Build a deep integration with the GNS3 platform via its REST API, enabling automatic topology discovery, node management, live console access, and configuration synchronization.
3. **Implement an AI-Powered Copilot:** Design and implement an agentic AI assistant powered by Google's Gemini large language model, equipped with 34 callable tools that enable autonomous network discovery, configuration generation, deployment, auditing, and connectivity tracing.

4. **Enable Multi-Protocol Device Communication:** Support multiple communication protocols (Telnet, SSH, Serial) with robust error handling, including a novel raw TCP approach for reliable GNS3 console access.
5. **Support Multi-Vendor Extensibility:** Design a vendor-agnostic architecture using the strategy pattern, currently supporting Cisco IOS and Huawei VRP, with the ability to add new vendor profiles without modifying core logic.

1.4 SCOPE AND LIMITATIONS

ANCS is designed as a desktop application targeting the following scope:

In Scope:

- Cisco IOS configuration for routers, Layer 3 core switches, and Layer 2 access switches
- Huawei VRP configuration as a secondary vendor profile
- GNS3 integration for network simulation environments
- AI Copilot with Gemini, OpenRouter, and Ollama provider support
- Windows and Linux desktop platforms
- Routing protocols: OSPF, EIGRP, RIPv2, and static routing
- Layer 2 features: VLANs, trunking (802.1Q), STP, inter-VLAN routing (SVI and router-on-a-stick)
- Services: DHCP server pools, standard and extended ACLs

Limitations:

- Physical device deployment requires SSH or serial connectivity; ANCS does not include SNMP-based management
- The application is not a cloud-hosted SaaS solution; it runs locally on the user's machine

- Real-time traffic analysis and monitoring are not included in the current version
- Only Cisco IOS and Huawei VRP vendor profiles are implemented; Juniper JunOS and others are planned for future work

1.5 PROJECT SIGNIFICANCE

ANCS addresses a critical need in both educational and professional networking contexts:

Educational Value: For networking students using GNS3 as their primary learning tool, ANCS transforms the simulation experience. Instead of spending hours on repetitive CLI commands, students can use guided wizards to generate configurations and focus on understanding networking concepts. The AI Copilot serves as an intelligent tutor that can explain configurations, audit networks for best-practice violations, and trace connectivity paths — capabilities that enhance the learning experience beyond what any current tool provides [9].

Professional Value: For network administrators managing small to medium-sized networks, ANCS provides a zero-code automation solution. The guided wizard approach eliminates the need for Python or YAML expertise, making network automation accessible to administrators of all skill levels.

Technical Innovation: ANCS introduces several novel technical approaches: (1) raw TCP communication with IAC byte stripping for reliable GNS3 console access, solving a long-standing reliability issue; (2) an agentic AI system with deterministic safety guardrails that prevent the LLM from deploying harmful configurations; and (3) a hybrid desktop/web

architecture using QWebEngineView for rich, modern UI rendering within a native application framework.

1.6 METHODOLOGY

The development of ANCS followed an Agile methodology with iterative prototyping. The project was divided into weekly sprints, each targeting specific functional milestones:

6. **Sprint 1–3:** Core infrastructure — PySide6 application shell, SQLite database schema, GNS3 REST API client, and basic device management UI.
7. **Sprint 4–6:** Network communication — Telnet, SSH, and Serial protocols; raw TCP implementation for GNS3; IAC byte stripping; console wake-up protocol.
8. **Sprint 7–9:** Configuration engine — Vendor profile architecture, ConfigEngine, Guided Setup Wizard, and preset templates.
9. **Sprint 10–13:** AI Copilot — Gemini integration, tool schema generation, 34 tools implementation, safety guardrails, and QWebEngineView chat HUD.
10. **Sprint 14–16:** Polish and testing — Live state synchronization, IOS parser, bulk deployment, subnet calculator, PDF report generation, and comprehensive testing.

1.7 BOOK ORGANIZATION

The remainder of this book is organized as follows:

Chapter 2: Theoretical Background and Related Work provides a comprehensive survey of network automation concepts, existing tools,

GNS3 simulation, AI in network management, and desktop application frameworks. A gap analysis positions ANCS relative to existing solutions.

Chapter 3: Methodology — System Design and Implementation details the system requirements, architecture, UML diagrams, database design, and implementation of all major subsystems including the network communication layer, configuration engine, AI Copilot, and graphical user interface.

Chapter 4: Experiments and Results presents the testing strategy, test cases, performance metrics, AI evaluation results, and a feasibility study.

Chapter 5: Conclusion and Future Work summarizes achievements, key contributions, challenges encountered, and outlines future development directions.

CHAPTER TWO

THEORETICAL BACKGROUND AND RELATED WORK

2.1 NETWORK AUTOMATION OVERVIEW

Network automation refers to the process of automating the configuration, management, testing, deployment, and operation of network devices using software. The evolution of network automation can be traced through several distinct phases, each representing a significant advancement in how network infrastructure is managed [10].

Phase 1: Manual CLI Configuration (1990s–2000s). The earliest approach to network management involved administrators connecting to each device via console cable or Telnet and manually entering configuration commands. This approach, while providing full control, is inherently unscalable. For a network of N devices, the configuration effort grows linearly, and the probability of human error increases with each additional device.

Phase 2: Script-Based Automation (2000s–2010s). The introduction of scripting languages — particularly Expect scripts, Perl, and later Python — enabled administrators to automate repetitive CLI interactions. Libraries such as Pexpect and, later, Paramiko provided programmatic SSH access to network devices. However, these scripts were fragile, device-specific, and required significant programming expertise.

Phase 3: Framework-Based Orchestration (2010s–2020s). Purpose-built automation frameworks emerged to address the limitations of ad-hoc scripting. Ansible (2012), with its agentless, YAML-based approach, became the most popular tool for network automation. NAPALM (2015) provided a unified API across multiple vendor platforms. These frameworks introduced concepts such as idempotency, inventory management, and declarative configuration [11].

Phase 4: AI-Assisted Automation (2020s–Present). The emergence of large language models (LLMs) has introduced a new paradigm: AI-assisted network management, often termed AIOps (Artificial Intelligence for IT Operations). LLMs can understand natural language queries about network configurations, generate device-specific CLI commands, and even reason about network topologies. This phase represents the current frontier, where ANCS positions itself as a pioneering implementation [12].

2.2 CISCO IOS ARCHITECTURE

Cisco IOS (Internetwork Operating System) is the operating system that runs on the majority of Cisco routers and switches. Understanding its architecture is essential for any network automation tool that targets Cisco devices [2].

2.2.1 Configuration Modes

Cisco IOS operates in a hierarchical mode structure:

- **User EXEC Mode (>):** Limited monitoring commands; the initial mode after login.
- **Privileged EXEC Mode (#):** Full monitoring and management commands; accessed via the `enable` command.

- **Global Configuration Mode (config)#:** System-wide configuration; accessed via `configure terminal`.
- **Interface Configuration Mode (config-if)#:** Per-interface settings.
- **Router Configuration Mode (config-router)#:** Routing protocol settings.

ANCS's Sender module must navigate these modes programmatically, detecting the current mode from the command prompt pattern (e.g., `Router#`, `Router(config)#`) and issuing the appropriate commands to transition between modes.

2.2.2 Running Configuration vs. Startup Configuration

IOS maintains two distinct configurations: the *running-config* (active in RAM, lost on reboot) and the *startup-config* (stored in NVRAM, loaded on boot). ANCS's ConfigPuller retrieves the running-config via the `show running-config` command, and the Sender can optionally save configurations using `copy running-config startup-config` or the Huawei VRP equivalent `save` command.

2.3 EXISTING AUTOMATION TOOLS

A comprehensive comparison of existing network automation tools provides context for ANCS's design decisions and highlights the gaps it addresses.

2.3.1 Ansible

Ansible, developed by Red Hat, is the most widely adopted network automation framework. It uses an agentless architecture that communicates with devices via SSH, and configurations are defined in YAML playbooks using Jinja2 templates. While powerful and extensible, Ansible requires

users to understand YAML syntax, Jinja2 templating, inventory file management, and module-specific parameters. The learning curve is significant for users without programming backgrounds [13].

2.3.2 Netmiko

Netmiko is a Python library that simplifies SSH connections to network devices. It extends Paramiko by adding device-specific logic for handling CLI prompts, configuration modes, and output parsing. Netmiko is a building block for custom automation scripts but provides no GUI, no configuration generation, and no AI assistance. ANCS uses a similar approach to Netmiko for SSH connections via Paramiko but extends it significantly with raw TCP support for GNS3, automated prompt detection, and block-based deployment [14].

2.3.3 NAPALM

NAPALM (Network Automation and Programmability Abstraction Layer with Multivendor support) provides a unified Python API for interacting with multiple network vendors. It supports configuration management, operational data retrieval, and configuration diffs. However, NAPALM is a library, not an application — it provides no user interface and requires Python programming to use [15].

2.3.4 SolarWinds Network Configuration Manager

SolarWinds NCM is a commercial tool that provides GUI-based network configuration management, backup, and compliance auditing. While feature-rich, it is expensive (licensing starts at thousands of dollars), targets enterprise environments, and does not support GNS3 simulation integration [16].

2.3.5 Cisco DNA Center

Cisco DNA Center is Cisco's enterprise network management platform that provides intent-based networking, automated provisioning, and analytics. It requires Cisco enterprise hardware, a dedicated server appliance, and enterprise licensing, making it inaccessible for educational use or small-scale deployments [17].

Table 2.1: Comparison of existing network automation tools.

Feature	ANCS	Ansible	Netmiko	NAPALM	SolarWinds	DNA Center
GUI Interface	✓	✗	✗	✗	✓	✓
No-Code Configuration	✓	✗	✗	✗	Partial	✓
GNS3 Integration	✓	✗	✗	✗	✗	✗
AI Assistant	✓	✗	✗	✗	✗	✗
Guided Wizards	✓	✗	✗	✗	✗	Partial
Multi-Vendor	✓	✓	✓	✓	✓	✗ (Cisco only)
Open Source	✓	✓	✓	✓	✗	✗
Cost	Free	Free	Free	Free	$$$$	$$$$$
Programming Required	No	YAML	Python	Python	No	No
Network Auditing	✓ (AI)	Custom	✗	Partial	✓	✓

2.4 GNS3 NETWORK SIMULATION PLATFORM

GNS3 (Graphical Network Simulator 3) is an open-source network emulation platform that allows users to design, build, and test network

topologies using real network device images. Unlike Cisco Packet Tracer, which simulates device behavior, GNS3 runs actual IOS images through Dynamips (for IOS) or QEMU/KVM (for IOSv, IOU, and other platforms), providing a faithful reproduction of real device behavior [5].

2.4.1 GNS3 Architecture

GNS3 operates with a client-server architecture:

- **GNS3 Server:** Manages the compute resources (Dynamips, QEMU, Docker containers) and exposes a REST API on port 3080 by default.
- **GNS3 Client:** The desktop GUI that communicates with the server via HTTP/REST API calls.
- **Console Access:** Each running node exposes a TCP console port (typically in the 5000–6000 range) that provides CLI access to the device.

ANCS replaces the GNS3 client role for configuration management, communicating directly with the GNS3 server via its REST API v2. The API endpoints used by ANCS include:

- `GET /v2/projects` — List all projects
- `GET /v2/projects/{id}/nodes` — List all nodes in a project
- `GET /v2/projects/{id}/links` — List all links
- `POST /v2/projects/{id}/nodes` — Create a new node
- `POST /v2/projects/{id}/links` — Create a link between nodes
- `POST /v2/projects/{id}/nodes/{id}/start` — Start a node
- `POST /v2/projects/{id}/drawings` — Add annotations to the topology

2.4.2 Console Port Challenge

A significant technical challenge when automating GNS3 devices is the console port implementation. GNS3 exposes device consoles as raw TCP

sockets, but these sockets include Telnet protocol negotiation (IAC sequences) that can interfere with command-line interaction. Standard Telnet libraries (including Python's `telnetlib3`) perform option negotiation that generates IAC bytes, which are interpreted as garbage characters by the IOS CLI. This challenge is addressed in detail in Chapter 3 [18].

2.5 ARTIFICIAL INTELLIGENCE IN NETWORK MANAGEMENT

The application of AI to network management — commonly termed AIOps (Artificial Intelligence for IT Operations) — represents one of the most significant trends in modern network engineering [12].

2.5.1 Large Language Models for Networking

Large Language Models (LLMs) such as Google's Gemini, OpenAI's GPT series, and Meta's Llama have demonstrated remarkable capability in understanding and generating domain-specific text, including network device configurations. These models can:

- Parse natural language requests ("Configure OSPF on all routers") into specific CLI commands
- Analyze existing configurations for security vulnerabilities or misconfigurations
- Explain complex network concepts in accessible language
- Generate complete configuration templates from high-level specifications

2.5.2 Agentic AI Architecture

The agentic AI paradigm extends LLMs beyond simple text generation by equipping them with *tools* — callable functions that the LLM can invoke to

interact with external systems. This architecture follows the **Thought → Action → Observation** loop [19]:

11. **Thought:** The LLM reasons about the user's request and determines the next step.
12. **Action:** The LLM generates a function call (tool invocation) with specific parameters.
13. **Observation:** The tool executes, and its result is fed back to the LLM as context.
14. **Repeat:** The LLM uses the observation to decide the next action or generate a final response.

Google's Gemini API natively supports this pattern through its function-calling feature, where tool definitions are provided as JSON schemas in the API request, and the model responds with structured function call objects rather than plain text when it determines a tool should be used [20]. ANCS implements this pattern with 34 tools, making it one of the most tool-rich agentic AI applications in the networking domain.

2.5.3 Safety Considerations

A critical concern when deploying AI agents in network environments is safety. Unlike text generation, network configuration changes can have immediate, real-world consequences — a misconfigured access control list can lock out legitimate users, and an incorrect routing statement can create network loops. ANCS implements multiple layers of safety guardrails, detailed in Chapter 3, including hostname verification, configuration provenance checking, and human-in-the-loop (HITL) interception for destructive operations [21].

2.6 DESKTOP APPLICATION FRAMEWORKS

The choice of desktop application framework significantly impacts the user experience, development productivity, and cross-platform compatibility. Three frameworks were evaluated for ANCS:

PySide6 (Qt 6 for Python) provides Python bindings for the Qt 6 framework, one of the most mature and comprehensive GUI toolkits available. Key advantages include native rendering on all platforms, extensive widget library, QWebEngineView for embedded web content, QThread for background operations, and a signal/slot mechanism for thread-safe UI updates. PySide6 was selected for ANCS because it combines the rapid development speed of Python with the performance and richness of Qt [22].

Electron uses web technologies (HTML, CSS, JavaScript) within a Chromium wrapper. While Electron provides excellent UI flexibility, it consumes significantly more memory (typically 100–300 MB base overhead) and requires a different technology stack from ANCS's Python-based networking code [23].

Tkinter is Python's built-in GUI toolkit. While simple and lightweight, it lacks modern UI components, web view embedding, and the rich styling capabilities needed for ANCS's glassmorphic design aesthetic [24].

2.7 MULTI-VENDOR NETWORK OPERATING SYSTEMS

Modern networks increasingly include equipment from multiple vendors. While Cisco IOS remains dominant, Huawei's VRP (Versatile Routing

Platform) has gained significant market share globally, particularly in enterprise and carrier networks.

The fundamental challenge of multi-vendor automation is that each vendor uses different command syntax for equivalent operations. For example:

```
! Cisco IOS — Create VLAN 10
vlan 10
 name Staff

! Huawei VRP — Create VLAN 10
vlan 10
 description Staff
```

ANCS addresses this challenge through a *strategy pattern* architecture, where vendor-specific command generation is encapsulated in interchangeable profile classes. This approach allows the core configuration logic to remain vendor-agnostic while delegating syntax generation to the appropriate vendor profile, as detailed in Section 3.7.1.

2.8 GAP ANALYSIS AND ANCS POSITIONING

Based on the analysis of existing tools and technologies, the following gaps are identified:

Table 2.2: Gap analysis — ANCS vs existing tools.

Gap	Description	How ANCS Addresses It
No-Code Automation	All open-source tools require programming	Guided wizards generate configs from form inputs
GNS3 Automation	No existing tool automates GNS3 deployments	Full GNS3 REST API integration with topology discovery
AI + Guardrails	AI assistants lack deployment capabilities and safety mechanisms	34-tool agentic AI with hostname, provenance, and HITL checks
Live State Sync	No tool auto-detects existing	ConfigPuller + IOSParser + drift

	configs to prevent conflicts	detection in wizard
Educational Accessibility	Enterprise tools are expensive; open-source tools need programming	Free, open-source, GUI-based, with AI tutor capabilities

ANCS occupies a unique position in the network automation landscape by combining visual topology management, guided configuration generation, and agentic AI assistance in a single, free, open-source desktop application. No existing tool — commercial or open-source — provides this combination of capabilities.

CHAPTER THREE

METHODOLOGY — SYSTEM DESIGN AND IMPLEMENTATION

This chapter presents the comprehensive design and implementation of ANCS. It begins with system requirements analysis, proceeds through architectural design and UML modeling, and details the implementation of each major subsystem with relevant code snippets and screenshots.

3.1 SYSTEM REQUIREMENTS

3.1.1 Functional Requirements

Table 3.1: Functional requirements.

ID	Requirement	Description
FR-01	GNS3 Topology Discovery	The system shall automatically discover projects, nodes, and links from a GNS3 server via REST API and store device information in the local database.
FR-02	Guided Configuration Wizard	The system shall provide a step-by-step wizard that generates complete Cisco IOS configurations for VLANs, routing protocols, DHCP, and ACLs based on user input and device role (router, core switch, access switch).
FR-03	Multi-Protocol Deployment	The system shall deploy configurations to devices via Telnet, SSH, or Serial connections, with automatic protocol selection based

		on device type and availability.
FR-04	AI Copilot	The system shall provide an AI assistant with at least 30 callable tools for autonomous network management, including topology discovery, configuration generation, deployment, auditing, and connectivity tracing.
FR-05	Live Configuration Pull	The system shall connect to live devices, retrieve their running configurations, parse them into structured data, and use this data to pre-populate the configuration wizard.
FR-06	Network Auditing	The system shall analyze device configurations for security vulnerabilities, including missing enable secrets, unprotected VTY lines, and VLAN inconsistencies.
FR-07	Connectivity Tracing	The system shall perform hop-by-hop path tracing between network endpoints by querying routing tables on each device in the path.
FR-08	Bulk Deployment	The system shall support concurrent deployment of configurations to multiple devices with staggered connections and per-device result reporting.
FR-09	Subnet Calculator	The system shall provide a VLSM subnet calculator that allocates subnets from a parent network based on department host requirements.
FR-10	PDF Report Generation	The system shall generate professional PDF reports summarizing network state, audit findings, or deployment results.

3.1.2 Non-Functional Requirements

Table 3.2: Non-functional requirements.

ID	Requirement	Description
NFR-01	Thread Safety	All database operations must use a global mutex lock to prevent concurrent access corruption. The database shall use WAL (Write-Ahead Logging) journal mode for safe concurrent reads.
NFR-02	UI Responsiveness	All network operations and AI queries must execute in background threads (QThread) to prevent UI freezing. UI updates from background threads must use Qt Signals.
NFR-03	Graceful Error Handling	The system shall handle GNS3 server disconnections, device timeouts, and API errors gracefully with user-friendly error messages.
NFR-04	LLM Cost Optimization	The AI Copilot shall implement context window compression and history truncation to minimize token usage and API costs.
NFR-05	Cross-Platform Compatibility	The application shall run on Windows 10+ and Ubuntu 20.04+ with identical functionality.

3.2 SYSTEM ARCHITECTURE

ANCS follows a three-tier layered architecture that separates concerns between the user interface, business logic, and data/network access layers. *Figure 3.1* illustrates the high-level architecture.

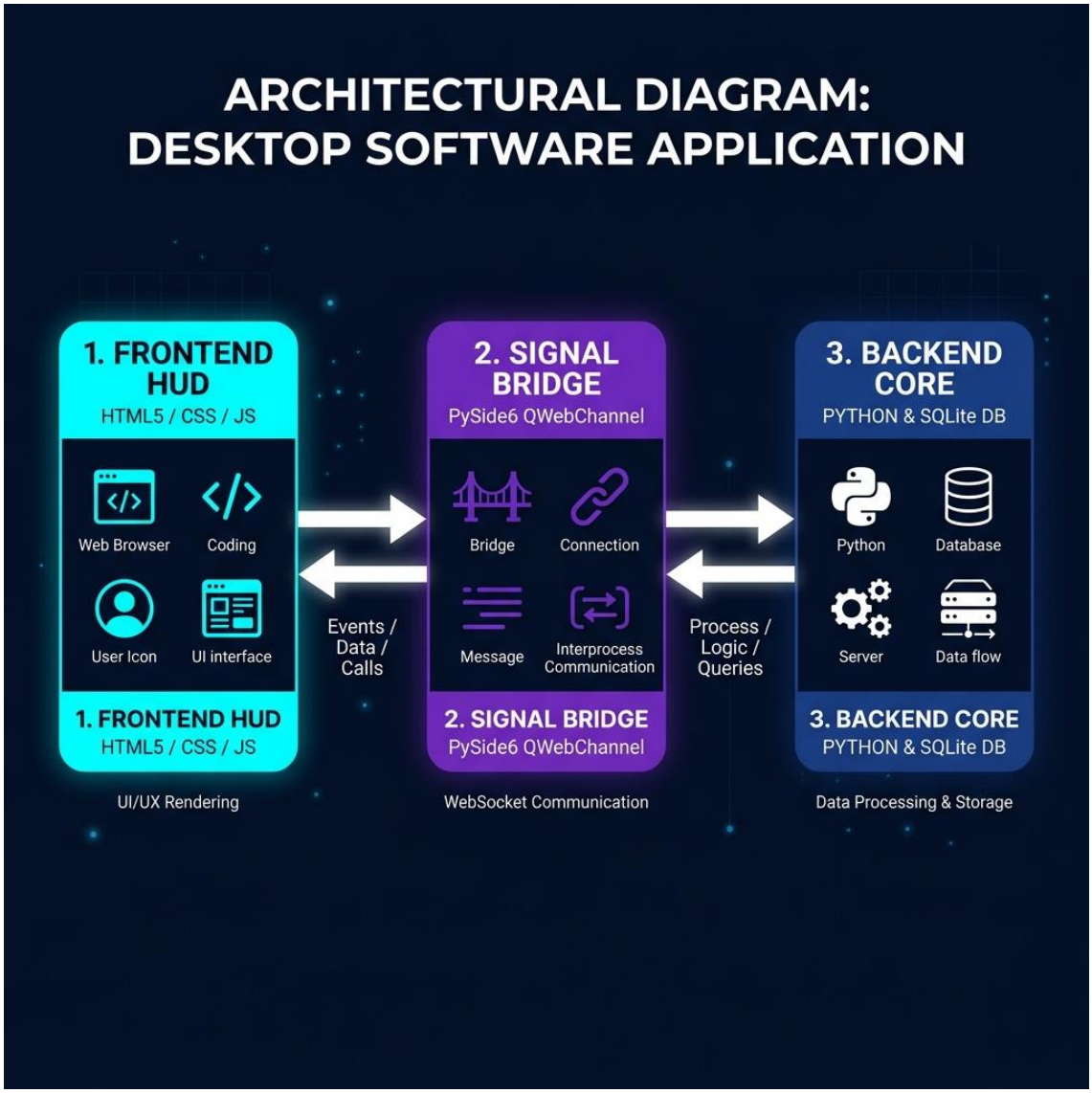

Figure 3.1: ANCS three-tier system architecture.

3.2.1 Presentation Layer

The presentation layer combines PySide6's native widget framework with an embedded web view:

- **Main Application Window (`app.py`):** A frameless PySide6 `QMainWindow` with custom window controls, glassmorphic CSS styling, and a device management grid.
- **AI Copilot HUD (`agent_dialog.py` + `web/index.html`):** A `QWebEngineView` that loads a self-contained HTML/CSS/JS application for the AI chat

interface. Bidirectional communication is achieved via `QWebChannel`, which establishes a bridge object (`AgentBridge`) accessible from both Python and JavaScript.

- **Guided Wizard (`guided_setup_wizard.py`):** A multi-step `QDialog` with dynamic form generation based on device role and configuration presets.

3.2.2 Business Logic Layer

The business logic layer contains the core intelligence of the system:

- **Configuration Engine (`config_engine.py`):** Transforms structured data into vendor-specific CLI commands using the strategy pattern with `VendorProfile` implementations.
- **AI Agent (`ai_agent.py`):** Manages the LLM conversation loop, tool dispatch, safety guardrails, and context window management.
- **Vendor Profiles (`vendors/`):** Encapsulate vendor-specific command syntax (Cisco IOS, Huawei VRP) behind a common interface.

3.2.3 Data and Network Access Layer

The bottom layer handles persistence and device communication:

- **SQLite Database:** 13 tables storing devices, configurations, credentials, chat history, snapshots, and operational logs. Uses WAL journal mode and a global `threading.Lock()` for thread safety.
- **Network Sender (`sender.py`):** Multi-protocol communication engine supporting Telnet (via `telnetlib3`), SSH (via `paramiko`), and Serial (via `pyserial`), with a custom raw TCP mode for GNS3 console access.
- **GNS3 Connector (`gns3.py`):** REST API client wrapping `requests` for all GNS3 server interactions.

- **Configuration Puller (`puller.py`):** Connects to devices to retrieve and parse running configurations.

3.3 UML DIAGRAMS

3.3.1 Use Case Diagram

The use case diagram identifies three actors interacting with the ANCS system:

- **Network Engineer (Primary Actor):** The human user who interacts with the GUI, uses the wizard, queries the AI Copilot, and initiates deployments.
- **AI Copilot (Autonomous Agent):** An intelligent agent that can autonomously discover network topology, generate configurations, deploy to devices, and audit the network.
- **GNS3 Server (External System):** The simulation platform that provides topology data and console access to virtual devices.

Key use cases include:

15. Synchronize GNS3 Topology
16. Configure Device via Wizard
17. Deploy Configuration to Device
18. Pull Live Configuration
19. Query AI Copilot
20. Audit Network Security
21. Trace Network Connectivity
22. Generate PDF Report
23. Bulk Deploy to Multiple Devices
24. Calculate Subnets (VLSM)
25. Manage Device Credentials

26. Provision Complete Topology

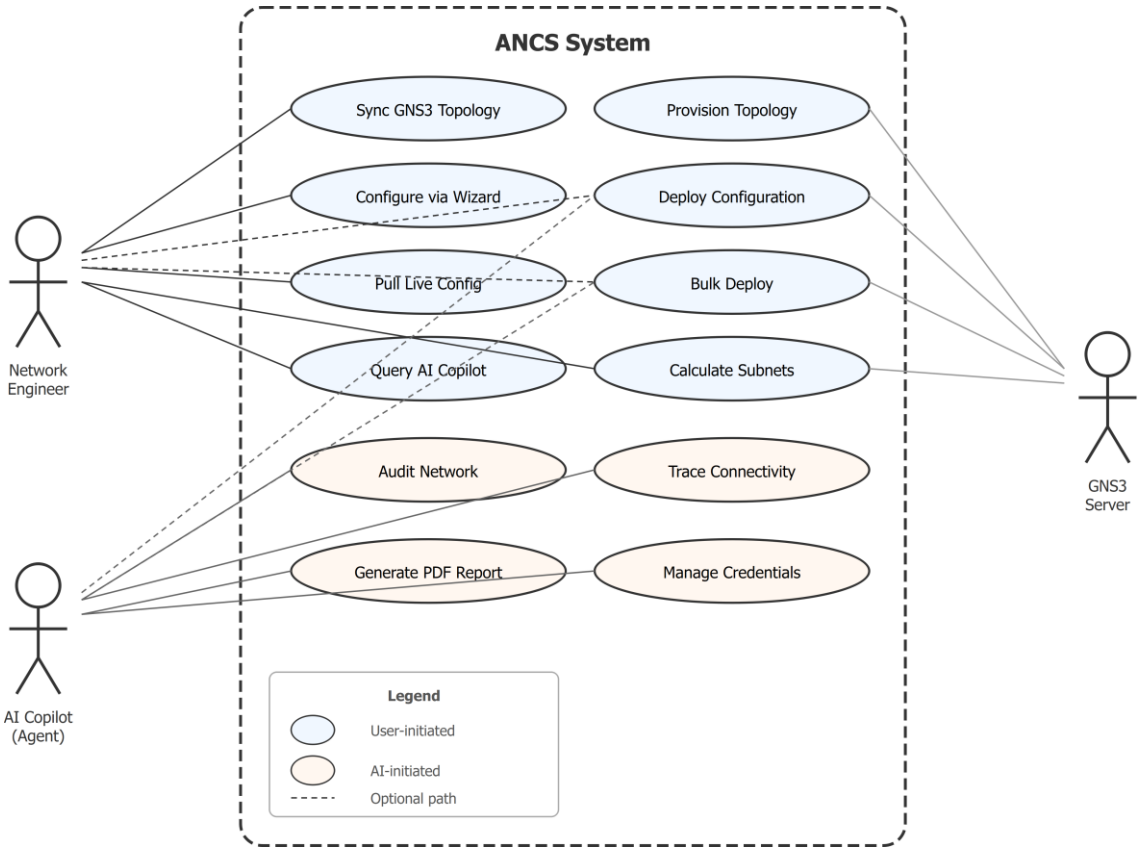

Figure 3.2: Use case diagram showing actors and system use cases.

3.3.2 Class Diagram

The class diagram captures the core domain classes and their relationships. The key class hierarchies are:

Device Model Hierarchy: An abstract `DeviceModel` base class with three concrete implementations: `RouterModel`, `CoreSwitchModel` (Layer 3), and `SwitchModel` (Layer 2). Each model defines device-specific configuration capabilities and default interface patterns.

Vendor Profile Hierarchy: An abstract `VendorProfile` base class with `CiscoIOSProfile` and `HuaweiVRPProfile` implementations. Each profile provides methods for rendering identity blocks, VLAN configurations, routing

protocol commands, DHCP pools, ACL rules, and save commands in the vendor's specific syntax.

Core Service Classes:

- ConfigEngine — Shared configuration generator used by both the wizard and AI Copilot
- Sender — Static methods for multi-protocol device communication
- GNS3Connector — REST API client for GNS3 server interaction
- CopilotWorker(QThread) — Background thread managing the AI conversation loop
- AgentBridge(QObject) — QWebChannel bridge between Python and JavaScript
- _AgentContext — Singleton holding mutable AI agent state
- IOSParser — Regex-based Cisco IOS configuration parser
- ConfigPuller — Device configuration retrieval engine

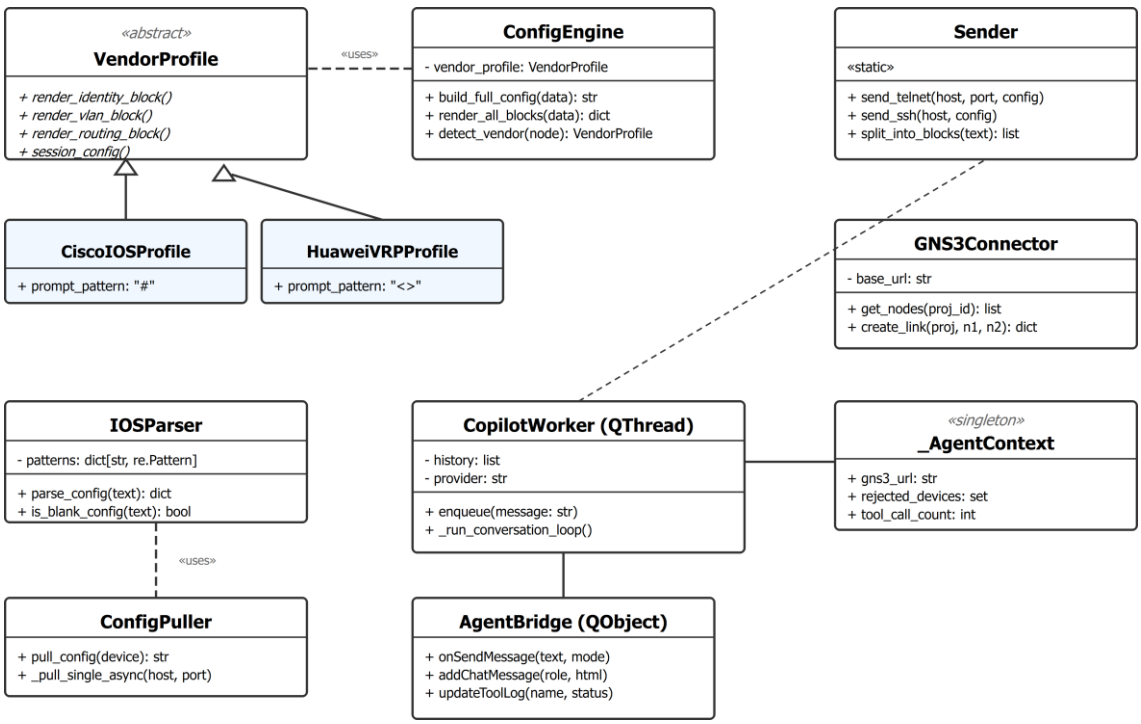

Figure 3.3: Class diagram showing core domain classes and their relationships.

3.3.3 Sequence Diagrams

GNS3 Topology Synchronization

When the user initiates a GNS3 sync, the following sequence occurs:

27. User clicks "Sync GNS3" in the main dashboard
28. `app.py` calls `GNS3Connector.get_nodes(project_id)`
29. `GNS3Connector` sends `GET /v2/projects/{id}/nodes` to the GNS3 server
30. The server returns a JSON array of node objects with name, type, console port, and status
31. `app.py` inserts or updates each node in the SQLite `devices` table
32. The UI grid is refreshed to display the discovered devices

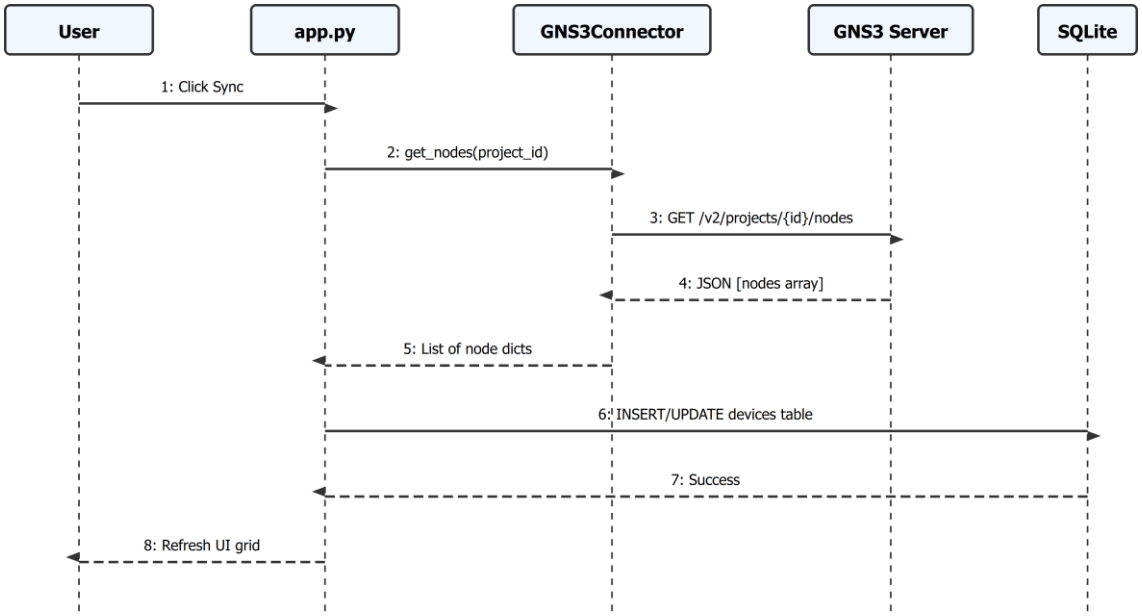

Figure 3.4: Sequence diagram for GNS3 topology synchronization.

Configuration Deployment Flow

The configuration deployment sequence from wizard to device involves:

33. User completes the Guided Setup Wizard and clicks "Deploy"
34. Wizard calls `ConfigEngine.build_full_config()` with structured data
35. `ConfigEngine` delegates to the appropriate `VendorProfile`

36. The vendor profile renders each configuration block (identity, VLANs, routing, DHCP, ACL)
37. `ConfigEngine` concatenates blocks with `! BLOCK N: Title` headers
38. `Sender.send_telnet()` opens a raw TCP connection to the device
39. `_telnet_wake_gns3_console()` handles the "Press RETURN" prompt
40. Login sequence executes (username/password if required, then `enable`)
41. `split_into_blocks()` partitions the config at block headers
42. Each block is sent line-by-line with `send_and_wait()`
43. Inter-block delays (3–4 seconds) prevent IOS parser overflow
44. Deployment result is logged to the database

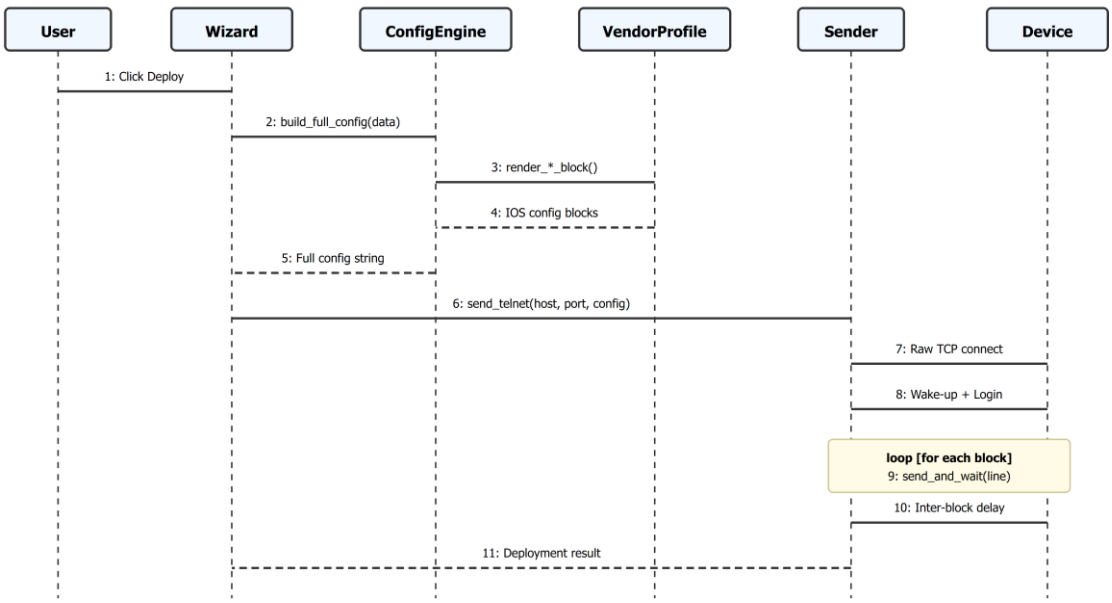

Figure 3.5: Sequence diagram for configuration deployment from wizard to device.

AI Copilot Query Flow

When a user sends a message to the AI Copilot, the following sequence occurs:

45. User types a message in the chat interface (JavaScript in `index.html`)
46. JavaScript calls `bridge.onSendMessage(text, mode)` via QWebChannel
47. `AgentBridge` receives the call in Python and emits `user_message_signal`

48. `CopilotWorker` enqueues the message and begins the LLM conversation loop
49. The message, system prompt, and conversation history are sent to the Gemini API
50. Gemini responds with either a text response or a `function_call`
51. If `function_call`: the tool function is dispatched and executed; its result is appended to the conversation and sent back to Gemini
52. The loop continues until Gemini produces a final text response
53. `CopilotWorker` emits `chat_response_signal`
54. `AgentBridge.addChatMessage()` is called, which invokes JavaScript to update the DOM

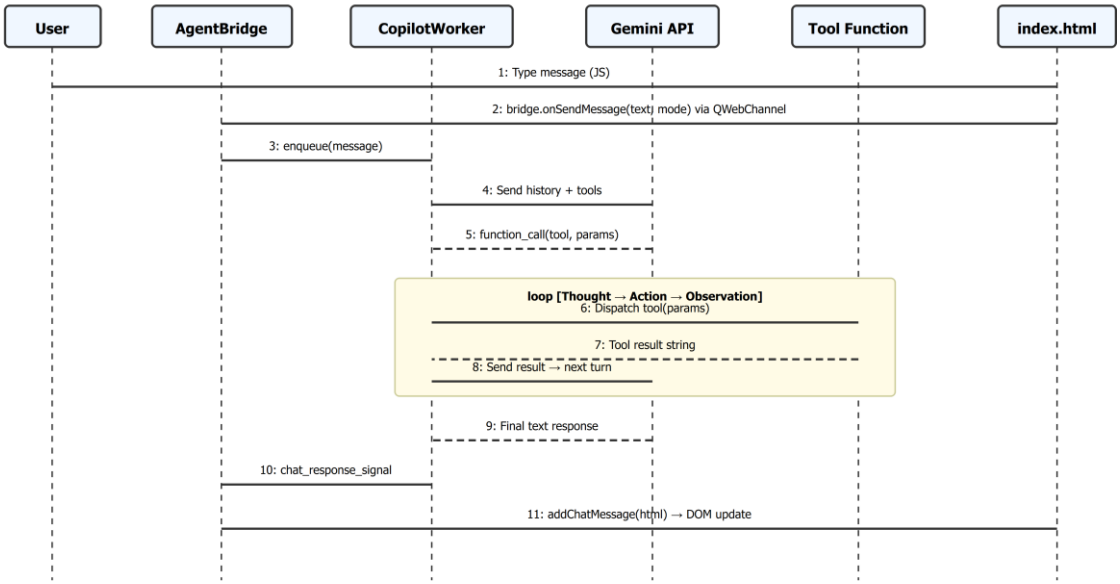

Figure 3.6: Sequence diagram for AI Copilot query and tool execution flow.

3.3.4 Activity Diagrams

Guided Setup Wizard Flow

The Guided Setup Wizard follows a role-aware, branching activity flow. The wizard first determines the device role (Router, Core Switch, or Access Switch), then presents appropriate configuration steps:

55. **Start:** User selects a device and opens the wizard
56. **Routing Mode Dialog:** Choose configuration preset (Small Office, School Lab, Minimal, Custom)
57. **Identity Step:** Hostname, domain name, enable secret, console/VTY passwords
58. **VLAN Step:** Define VLANs with names, IDs, and optional DHCP association
59. **[Decision: Device Role]** If Router → **Subinterface Step:** Configure 802.1Q subinterfaces for inter-VLAN routing If Core Switch → **SVI Step:** Configure Switched Virtual Interfaces with IPs If Access Switch → Skip (no Layer 3 interfaces)
60. **DHCP Step:** Configure DHCP pools for selected VLANs
61. **Routing Step:** Select and configure routing protocol (OSPF, EIGRP, RIP, Static)
62. **[Decision: Boundary Router?]** If yes → **Redistribution Step:** Configure mutual redistribution between protocols
63. **ACL Step:** Define standard and extended ACLs
64. **Summary Step:** Review generated configuration
65. **Deploy:** Send configuration to device

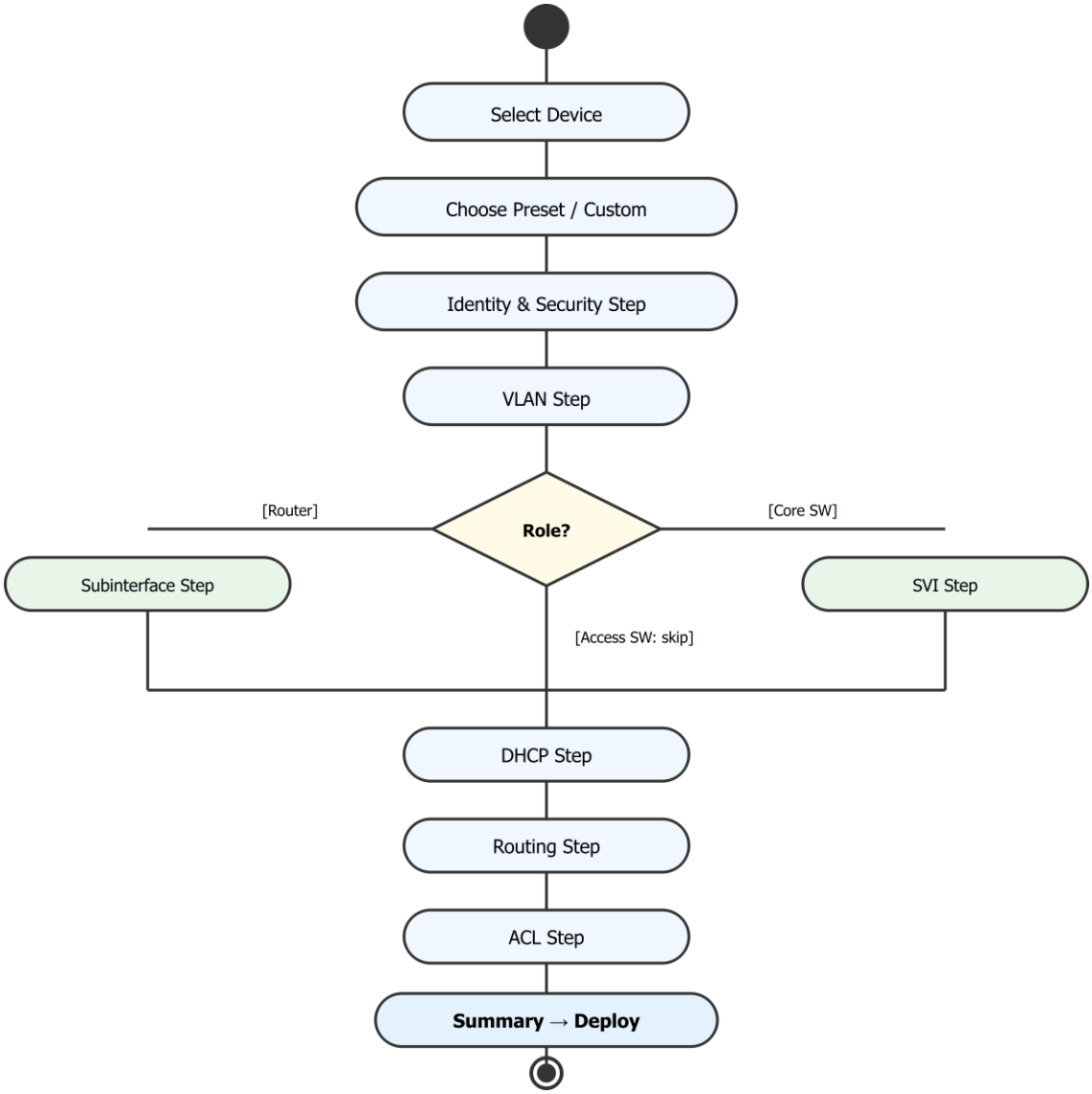

Figure 3.7: Activity diagram for the Guided Setup Wizard showing role-based branching.

3.4 DATABASE DESIGN

ANCS uses SQLite as its persistence layer, chosen for its zero-configuration deployment, file-based storage, and adequate performance for a single-user desktop application. The database employs WAL (Write-Ahead Logging) journal mode, which allows concurrent reads while a write is in progress — essential for ANCS where the AI Copilot thread and the GUI thread may both access the database simultaneously [25].

Thread safety is enforced through a global `threading.Lock()` object (`db_lock` in `config.py`) that must be acquired before any database write operation:

```
from config import db_lock

with db_lock:
    cursor.execute("INSERT INTO devices (...) VALUES (...)")
```

Table 3.3: Database tables summary.

Table	Purpose	Key Columns
devices	Network device inventory	name, type, ip, port, gns3_node_id, status
configs	Configuration templates/snapshots	device_name, config_text, timestamp
credentials	Device authentication credentials	device_name, username, password (Base64), enable_password
logs	Deployment and operation logs	device_name, action, result, timestamp
tasks	Background task tracking	task_type, status, progress, result
users	Application user profiles	username, role, preferences
device_configs	Device-specific configuration data	device_id, config_type, data
ai_models	AI model provider configurations	provider, model_name, api_key, is_active
training_data	AI training examples	input, output, category
api_keys	External API key storage	service, key, is_active
snapshots	Configuration version snapshots	device_name, config_text, snapshot_time
chat_conversations	AI chat session metadata	title, created_at, model_used
chat_messages	Individual chat messages	conversation_id, role, content,

		timestamp

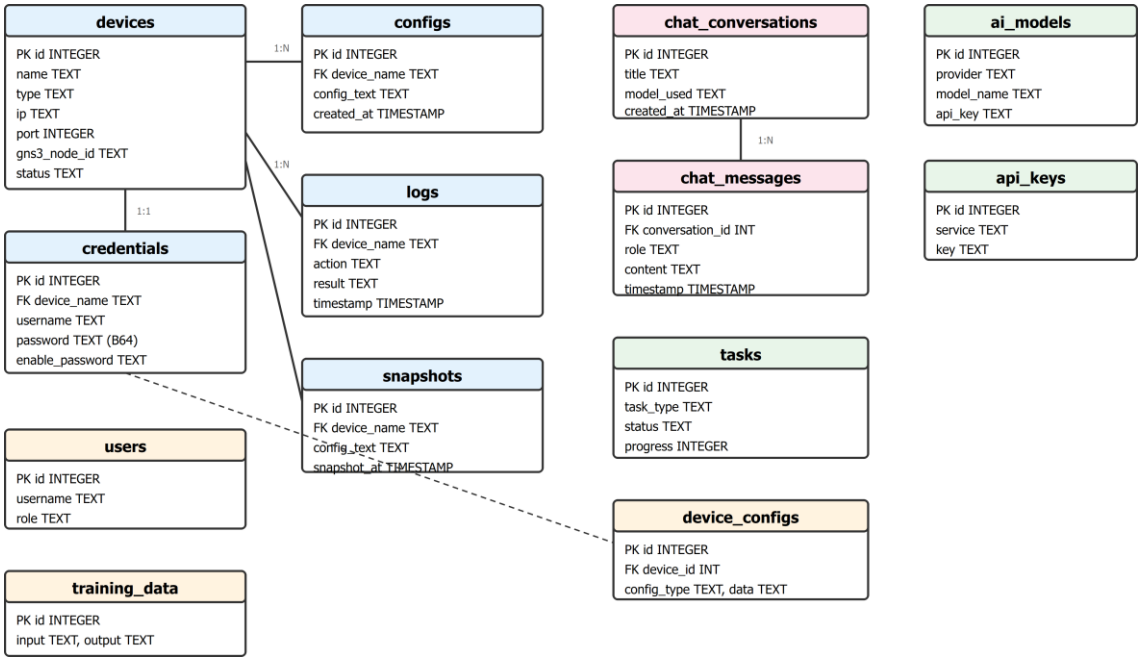

Figure 3.8: Entity-Relationship Diagram (ERD) showing database tables and relationships.

3.5 TECHNOLOGY STACK

Table 3.4: Technology stack.

Category	Technology	Version	Purpose
Language	Python	3.10+	Core application language
GUI Framework	PySide6 (Qt 6)	6.x	Native desktop UI, QWebEngineView, QThread
Web UI	HTML/CSS/JS	—	AI Copilot chat interface (embedded via QWebEngineView)
JS Bridge	QWebChannel	—	Bidirectional Python ↔ JavaScript communication
Database	SQLite	3.x	Local persistence with WAL mode
Telnet	telnetlib3	2.x	Async Telnet client for GNS3

			device consoles
SSH	Paramiko	3.x	SSH client for physical/production devices
Serial	pyserial	3.x	Serial console communication fallback
AI (Primary)	google-genai	—	Google Gemini API with function calling
AI (Secondary)	openai	—	OpenRouter and Ollama compatibility layer
HTTP Client	requests	2.x	GNS3 REST API communication
Markdown	markdown	—	Chat message rendering
IP Math	ipaddress	stdlib	Subnet calculation and validation

3.6 NETWORK COMMUNICATION LAYER

The network communication layer is the foundation of ANCS's ability to deploy configurations to real and simulated network devices. This section details the novel approaches developed to solve the unique challenges of GNS3 console communication.

3.6.1 Raw TCP and IAC Byte Stripping

The most significant technical challenge encountered during development was reliable communication with GNS3 device consoles. GNS3 exposes each device's console as a TCP socket, but this socket implements a minimal Telnet server that sends IAC (Interpret As Command) bytes as part of the Telnet protocol negotiation [18].

The Problem: When using standard Telnet libraries (such as Python's `telnetlib3`), the library responds to IAC sequences with its own Telnet option

negotiations (e.g., TTYPE, NAWS). These negotiation bytes pass through GNS3's TCP proxy and reach the IOS device console, where they are interpreted as garbage characters. This causes the IOS parser to produce errors, corrupts command output, and makes automated interaction unreliable.

The Solution: ANCS implements a raw TCP communication mode that bypasses Telnet negotiation entirely. Instead of using `telnetlib3` 's standard reader/writer, ANCS uses Python's `asyncio.open_connection()` to establish a plain TCP socket and wraps it in custom `_RawReader` and `_RawWriter` classes. All incoming data is processed through a `_strip_telnet_iac()` function that removes IAC sequences before they reach the application logic.

The IAC stripping function handles three categories of sequences:

66. **3-byte commands:** IAC + WILL/WONT/DO/DONT + Option (e.g., `0xFF 0xFB 0x01`) — stripped entirely
67. **Sub-negotiation:** IAC + SB + ... + IAC + SE — stripped entirely (variable length)
68. **Escaped 0xFF:** IAC + IAC (a literal 0xFF byte) — replaced with a single 0xFF

```
def _strip_telnet_iac(data: bytes) -> bytes:
    """Strip Telnet IAC sequences from raw TCP data."""
    cleaned = bytearray()
    i = 0
    while i < len(data):
        if data[i] == 0xFF and i + 1 < len(data):
            next_byte = data[i + 1]
            if next_byte in (0xFB, 0xFC, 0xFD, 0xFE):
                i += 3  # Skip 3-byte: IAC + WILL/WONT/DO/DONT + option
            elif next_byte == 0xFA:
                # Sub-negotiation: skip until IAC SE
                end = data.find(b'\xff\xf0', i + 2)
                i = end + 2 if end != -1 else len(data)
            elif next_byte == 0xFF:
                cleaned.append(0xFF)  # Literal 0xFF
                i += 2
            else:
                i += 2  # Skip other 2-byte IAC commands
        else:
```

```
        cleaned.append(data[i])
        i                                   +=                                  1
    return bytes(cleaned)
```

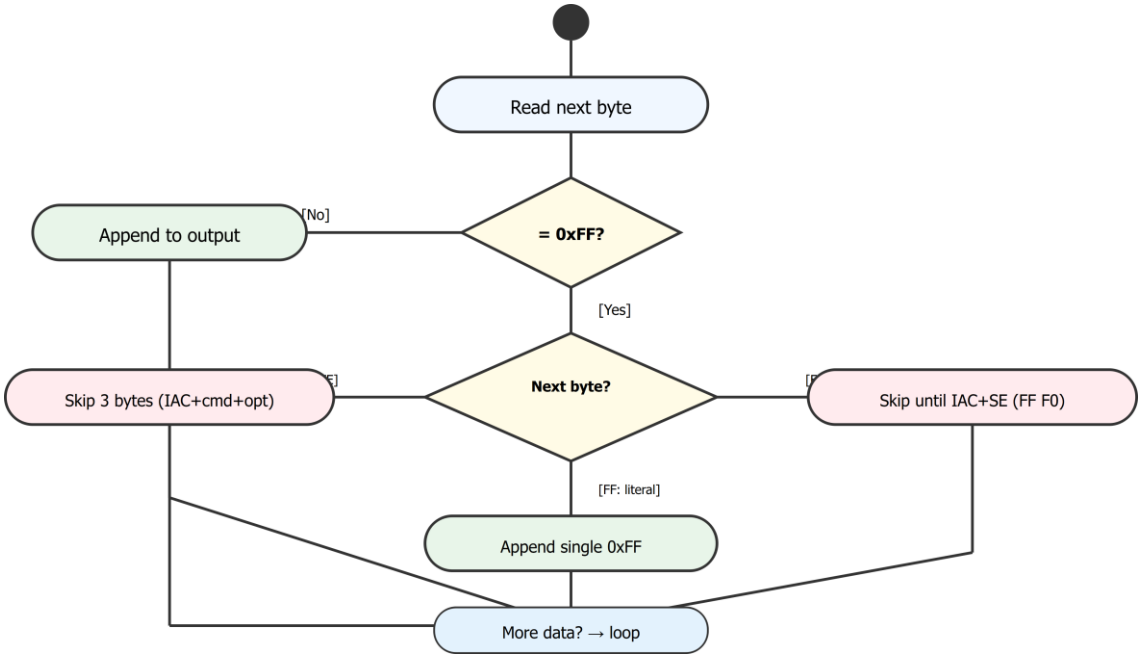

Figure 3.9: IAC byte stripping process flow showing how raw TCP data is cleaned.

3.6.2 GNS3 Console Wake-up Protocol

After establishing a raw TCP connection to a GNS3 device console, a second challenge arises: Cisco IOS devices display a "Press RETURN to get started" message after booting. The console may also contain boot-up log messages that must be drained before interactive use is possible.

ANCS implements a two-phase wake-up protocol:

Phase 1: Settling. The function `_telnet_wake_gns3_console()` performs up to 15 settling iterations, each sending a newline and reading with a short timeout (0.3–0.5 seconds). This drains any pending boot-up output, syslog messages, or buffered data from the console.

Phase 2: Prompt Detection. After settling, the function sends up to 12 additional newlines, each time reading the response and checking for

recognizable prompts: `Username:`, `Password:`, `#` (privileged mode), or `>` (user mode). Once a prompt is detected, the wake-up phase is complete and the connection is ready for command interaction.

3.6.3 Block-by-Block Configuration Deployment

ANCS uses a structured approach to configuration deployment that divides large configurations into manageable blocks. This is necessary because Cisco IOS devices have limited input buffer capacity, and sending hundreds of lines at once can overflow the parser, causing commands to be lost or misinterpreted.

Configurations generated by the `ConfigEngine` are formatted with explicit block headers:

```
! BLOCK 1: Identity & Security
hostname R1
enable secret class
...

! BLOCK 2: VLANs
vlan 10
 name Staff
...

! BLOCK 3: Routing
router ospf 1
 network 10.0.0.0 0.0.0.255 area 0
...
```

The `Sender.split_into_blocks()` method parses these headers to partition the configuration into individual blocks. Each block is then deployed sequentially:

69. Send `configure terminal` to enter global configuration mode
70. Send each line of the block with `send_and_wait()`, which writes the line and waits for the next prompt before proceeding
71. After completing a block, send `end` to return to privileged mode

72. Wait an inter-block delay (configurable, default 3–4 seconds) to allow the IOS parser to process
73. Repeat for the next block

This approach ensures reliable deployment even for large configurations with 100+ lines, and provides natural checkpoints for error detection between blocks.

3.6.4 GNS3 REST API Client

The `GNS3Connector` class provides a Pythonic interface to the GNS3 server's REST API. It wraps the `requests` library with consistent error handling, converting `ConnectionError` exceptions into user-friendly `RuntimeError` messages.

Key operations include:

- `get_projects()` — Lists all GNS3 projects with their IDs, names, and statuses
- `get_nodes(project_id)` — Returns all nodes with console ports, types, and metadata
- `get_ports(project_id, node_id)` — Returns port details for link creation
- `create_node(project_id, template_id, name, x, y)` — Creates a new node from a template
- `create_link(project_id, node1_id, adapter1, port1, node2_id, adapter2, port2)` — Creates a link between two node ports
- `start_node(project_id, node_id)` / `stop_node()` — Power control
- `create_drawing(project_id, svg, x, y)` — Adds visual annotations to the topology canvas

3.7 CONFIGURATION ENGINE

3.7.1 Vendor Profile Architecture

ANCS supports multiple network equipment vendors through a strategy pattern implementation. The abstract base class `VendorProfile` defines the interface that all vendor profiles must implement:

```
class VendorProfile(ABC):
    @abstractmethod
    def render_identity_block(self, data: dict) -> str: ...

    @abstractmethod
    def render_vlan_block(self, vlans: list) -> str: ...

    @abstractmethod
    def render_routing_block(self, protocol: str, data: dict) -> str: ...

    @abstractmethod
    def render_dhcp_block(self, pools: list) -> str: ...

    @abstractmethod
    def render_acl_block(self, acls: list) -> str: ...

    @abstractmethod
    def render_save_command(self) -> str: ...

    @abstractmethod
    def session_config(self) -> dict: ...
```

Two concrete profiles are currently implemented:

- **CiscoIOSProfile:** Generates Cisco IOS-specific commands. For example, VLAN creation uses `vlan {id}` / `name {name}`, and OSPF configuration uses `router ospf {process_id}` / `network {ip} {wildcard} area {area}`.
- **HuaweiVRPProfile:** Generates Huawei VRP-specific commands. VLAN creation uses `vlan {id}` / `description {name}`, and OSPF uses `ospf {process_id}` / `area {area}` / `network {ip} {wildcard}`.

Vendor detection is automatic: the `detect_vendor(gns3_node)` function examines GNS3 node metadata (template name, node type, image filename) for vendor-identifying keywords (e.g., "cisco", "ios", "c7200" for Cisco; "huawei", "vrp", "ce" for Huawei). The `session_config()` method returns vendor-

specific session parameters including prompt patterns, privilege escalation commands, and pagination disable commands (e.g., `terminal length 0` for Cisco, `screen-length 0 temporary` for Huawei).

3.7.2 Configuration Engine Core

The `ConfigEngine` class serves as the central configuration generation service, shared by both the Guided Setup Wizard and the AI Copilot. It accepts structured data (dictionaries with identity info, VLAN lists, routing parameters, etc.) and delegates command generation to the appropriate vendor profile.

The `build_full_config()` method orchestrates the generation process:

74. Detect the vendor profile from device metadata
75. Call each `render_*_block()` method to generate block-specific configuration
76. Assign sequential `! BLOCK N: Title` headers to each non-empty block
77. Concatenate all blocks into a single deployment-ready string
78. Append the vendor-specific save command as the final block

3.7.3 Guided Setup Wizard

The Guided Setup Wizard (`guided_setup_wizard.py`, 2583 lines) is the primary user-facing configuration interface. It implements a multi-step dialog that adapts its steps based on the device role (router, core switch, or access switch).

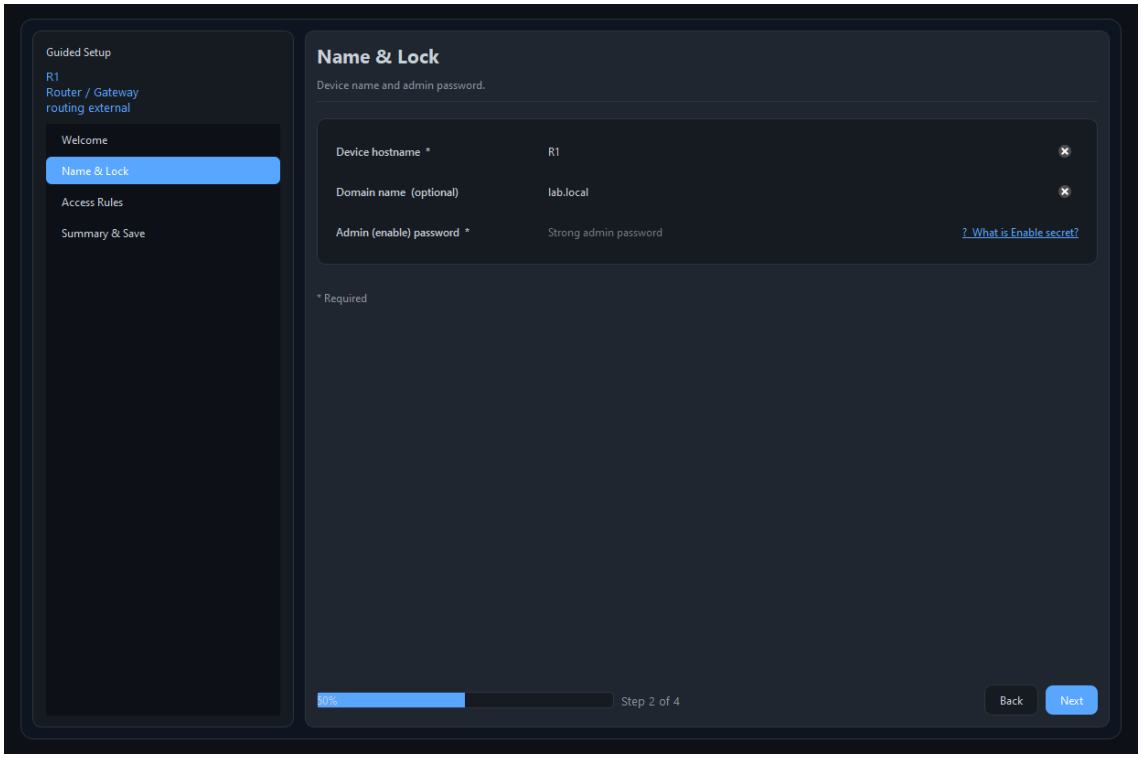

Figure 3.10: Guided Setup Wizard interface showing the VLAN configuration step.

Key features of the wizard include:

- **Configuration Presets:** Pre-built templates (Small Office, School Lab, Minimal) that auto-fill VLAN, routing, and security settings for common scenarios. The Small Office preset creates Staff (VLAN 10) and Guest (VLAN 20) networks with DHCP and a default route. The School Lab preset creates Students, Teachers, and Servers VLANs.
- **IP Collision Avoidance:** The `_pick_vlan_base()` method uses a set of known-used subnets (gathered from the topology) to select non-overlapping IP ranges for new VLANs.
- **Live Sync Integration:** When a device has an existing configuration (pulled by `ConfigPuller`), the wizard can pre-populate its forms from the parsed data. The `_get_sync_drift_tips()` method compares pulled config with wizard settings and generates user-visible tips about discrepancies.

- **Topology Awareness:** The `_find_link_to()` method resolves which local interface connects to a specific neighbor by querying GNS3 link data, enabling automatic interface selection for routing and trunking.

3.8 LIVE STATE SYNCHRONIZATION

ANCS provides live state synchronization capability through two cooperating modules: the Configuration Puller and the IOS Parser.

3.8.1 Configuration Puller

The `ConfigPuller` class connects to live devices to retrieve their running configurations. The pull process follows the same connection and wake-up protocol as configuration deployment:

79. Open raw TCP connection to device console port
80. Execute wake-up protocol (settling + prompt detection)
81. Login with stored credentials
82. Enter privileged mode (`enable`)
83. Disable pagination (`terminal length 0`)
84. Execute `show running-config`
85. Read output until the privileged prompt reappears
86. Strip command echo and trailing prompt from the output

The `is_blank_config()` method implements hyper-strict heuristics to determine if a device has a meaningful (non-default) configuration. It checks for:

- Manually configured IP addresses (excluding DHCP-assigned and link-local)
- Non-default hostnames (excluding patterns like "Router", "Switch", "R1", "SW1", "IOUx")
- Configured routing protocols (OSPF, EIGRP, RIP)

- Non-default VLANs (VLAN IDs outside the 1, 1002–1005 default range)
- Switched Virtual Interfaces (SVIs)

3.8.2 IOS Configuration Parser

The `IOSParser` class transforms raw IOS configuration text into structured data using 11 pre-compiled regular expressions. The parsed output is a Python dictionary that directly maps to the wizard's form fields, enabling seamless pre-population.

Parsed elements include:

- Hostname and domain name
- VLANs with names
- SVIs with IP addresses and subnet masks
- DHCP pools with network, gateway, and DNS settings
- Routing protocols with network statements and parameters
- Static routes
- WAN/uplink interface configurations

3.9 AI COPILOT IMPLEMENTATION

3.9.1 Architecture Overview

The AI Copilot is the most sophisticated subsystem of ANCS. It implements an agentic AI architecture where a large language model (LLM) autonomously decides which tools to invoke to fulfill user requests.

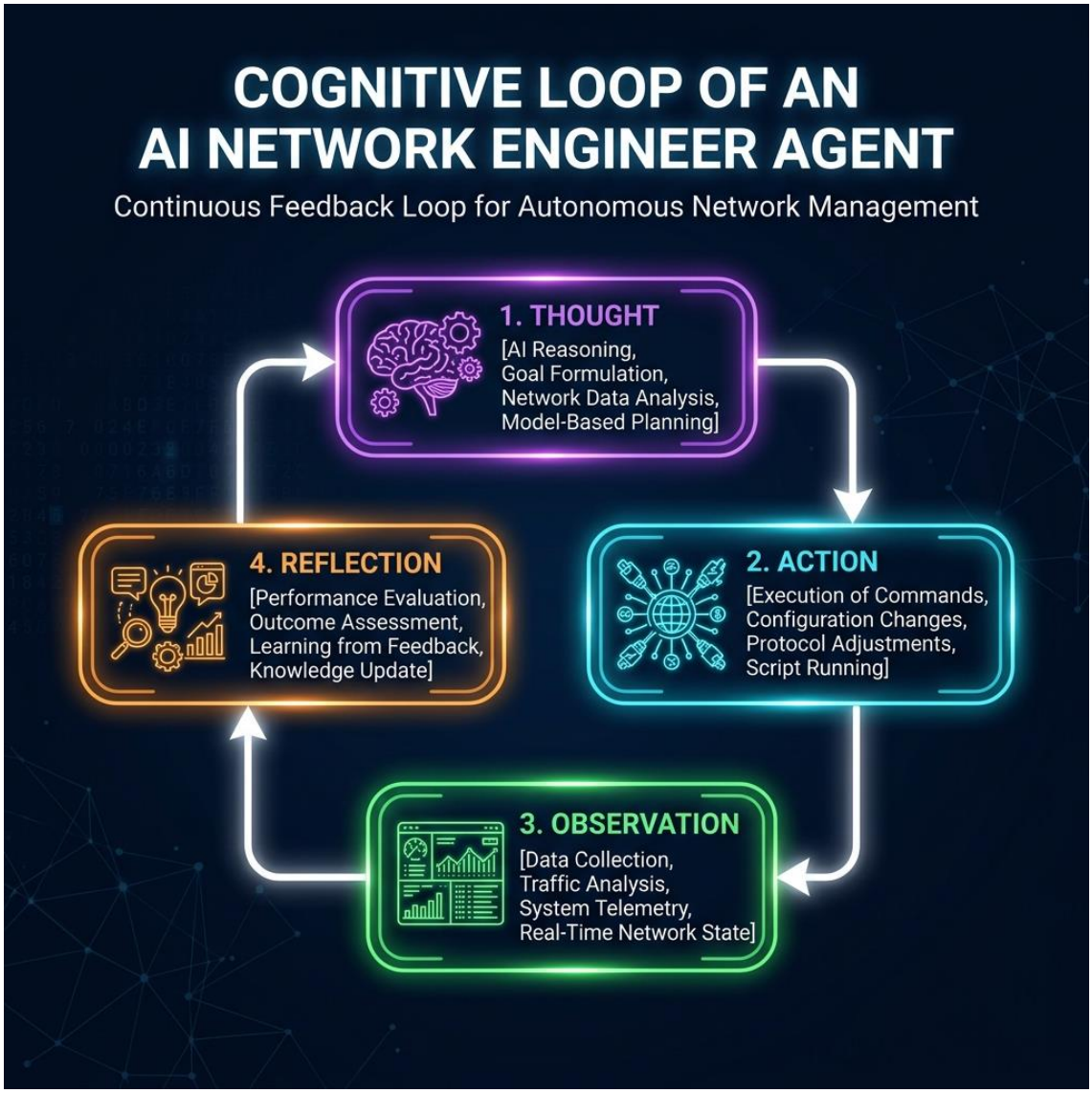

Figure 3.11: AI Copilot cognitive loop — Thought → Action → Observation cycle with safety guardrails.

The core components are:

- **CopilotWorker (QThread):** A background thread that manages the entire LLM conversation loop, preventing UI freezing during potentially long-running AI interactions.
- **_AgentContext (Singleton):** Holds mutable state including the GNS3 URL and project ID, a connection session pool, the current log function, safety counters (rejected devices, failure count), and tool execution statistics.

- **Provider Abstraction:** ANCS supports three AI providers: *Gemini* (via `google-genai`): The primary provider with native function calling *OpenRouter* (via `openai` SDK): Access to multiple models through a unified API *Ollama* (via `openai` SDK): Local model execution for offline/private use

3.9.2 Dynamic Tool Schema Generation

Rather than maintaining manual JSON schemas for each tool, ANCS generates tool definitions dynamically at startup using Python's `inspect` module. For each function in the `ALL_TOOLS` list, the system:

87. Extracts the function signature using `inspect.signature()`
88. Reads parameter names, type annotations, and default values
89. Parses the docstring to extract parameter descriptions
90. Constructs a Gemini-compatible function declaration with the function name, description, and parameter schema

This approach ensures that tool schemas always match their implementations, eliminating a common source of bugs in agentic AI systems where schemas and implementations can drift apart.

3.9.3 Tool Categories

The AI Copilot has access to 34 tools organized into 7 categories:

Table 3.5: AI Copilot tools by category.

Category	Count	Tools
GNS3 Discovery & Management	14	list_gns3_projects, list_gns3_nodes, list_gns3_templates, add_gns3_node, provision_topology, delete_gns3_node, connect_gns3_nodes, delete_gns3_link,

		control_gns3_node_power, move_gns3_node, add_gns3_annotation, clear_gns3_annotations, get_node_ports, get_topology_links
Terminal	1	run_cli_on_device
Database	5	list_all_devices, get_device_credentials, get_saved_config, get_send_history, query_logs
Configuration Generation	4	generate_device_config, generate_and_deploy_device_config, detect_topology, suggest_configs
Deployment	3	deploy_to_device, verify_device, bulk_deploy
Intelligence	5	snapshot_network_state, cleanup_device, audit_network, trace_connectivity, validate_configs
Utilities	3	calculate_subnet, get_agent_guidelines, generate_pdf_report

Notable tools include:

- **provision_topology:** An atomic operation that creates multiple nodes and links in a single invocation, enabling the AI to build entire network topologies from a natural language description (e.g., "Create a topology with 3 routers connected in a triangle").
- **snapshot_network_state:** Captures the "golden trio" of network state — interface status, ARP table, and routing table — from a device, providing comprehensive diagnostic data.
- **audit_network:** Scans all device configurations for security vulnerabilities including missing enable secrets, unprotected VTY lines, open console ports, and VLAN inconsistencies.

- **trace_connectivity:** Performs hop-by-hop path tracing by querying routing tables on successive devices, identifying the path a packet would take from source to destination.

3.9.4 Safety Guardrails

ANCS implements seven layers of safety to prevent the AI Copilot from deploying harmful configurations:

91. **Hostname Verification:** Before deploying to a device, the system verifies that the configuration's hostname matches the target device's name. A regex-based check prevents cross-device configuration deployment.
92. **Provenance Check:** Only configurations that were generated by ANCS's ConfigEngine or saved in the database can be deployed. This prevents the AI from deploying arbitrary text that might contain dangerous commands.
93. **Human-in-the-Loop (HITL) Interception:** When the AI uses `generate_and_deploy_device_config`, the configuration is first shown to the user in a visual diff view. Deployment only proceeds after explicit user approval.
94. **Device Rejection Tracking:** When a user rejects a deployment for a specific device, that device is added to a `rejected_devices` set. The AI is informed of the rejection and will not attempt to deploy to that device again in the same session.
95. **Consecutive Failure Abort:** If the AI encounters 5 consecutive tool failures, the system aborts the current operation and reports the issue to the user.
96. **Tool Call Counter:** The total number of tool calls is tracked per session to detect and prevent infinite loops.

97. **Token Usage and Cost Tracking:** Input tokens, output tokens, and estimated cost are tracked and displayed to the user, providing transparency about AI resource consumption.

3.9.5 Context Window Management

LLMs have limited context windows (typically 8K–128K tokens). For long interactions with many tool calls, the conversation history can exceed these limits. ANCS implements two compression strategies:

- **History Truncation (`_truncate_history()`):** Maintains a sliding window of the 20 most recent messages, preserving the system prompt and ensuring tool call/result pairs are never split (which would cause API errors).
- **Context Compression (`_compress_context()`):** For messages older than 6 turns, tool results are truncated to 200 characters. Individual tool results exceeding 10KB are truncated by `_truncate_tool_result()`.

3.9.6 Chat Interface (QWebEngineView HUD)

The AI Copilot's user interface is a self-contained web application (254KB HTML/CSS/JS) loaded in a `QWebEngineView`. This hybrid approach combines the flexibility and richness of web-based UI rendering with the native performance and system access of PySide6.

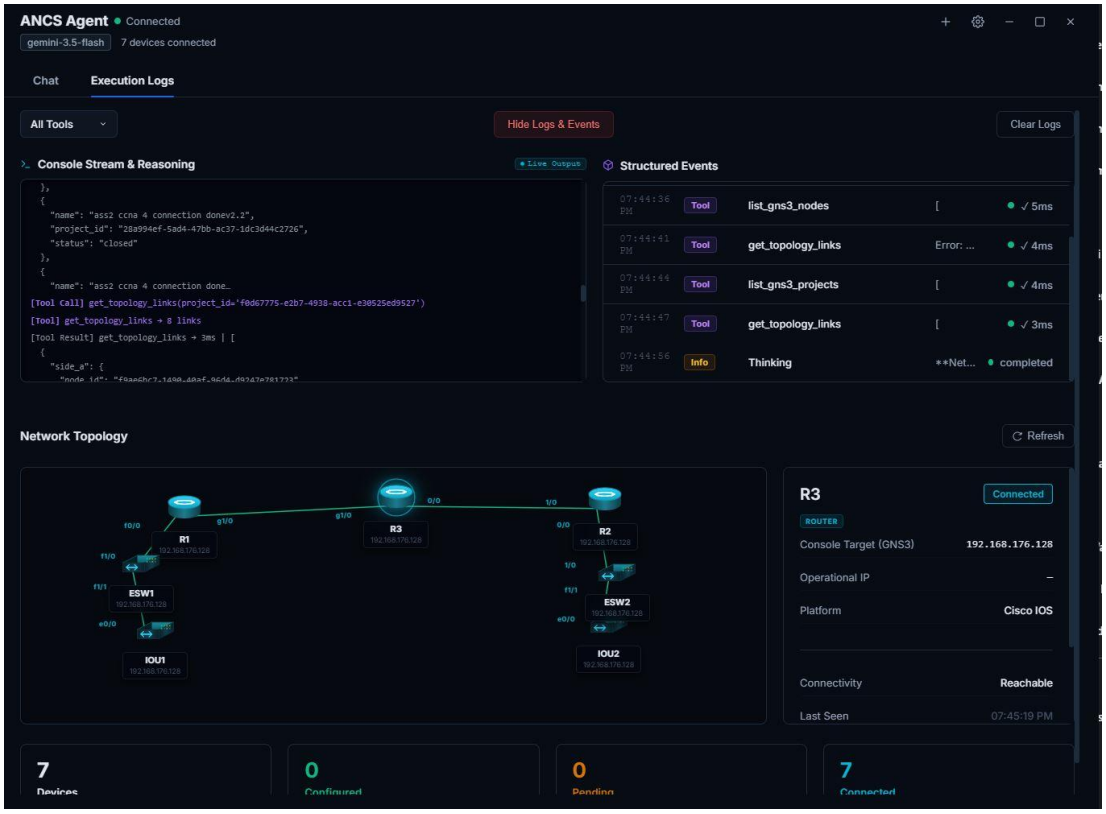

Figure 3.12: AI Copilot tool execution log showing timing and status for each tool call.

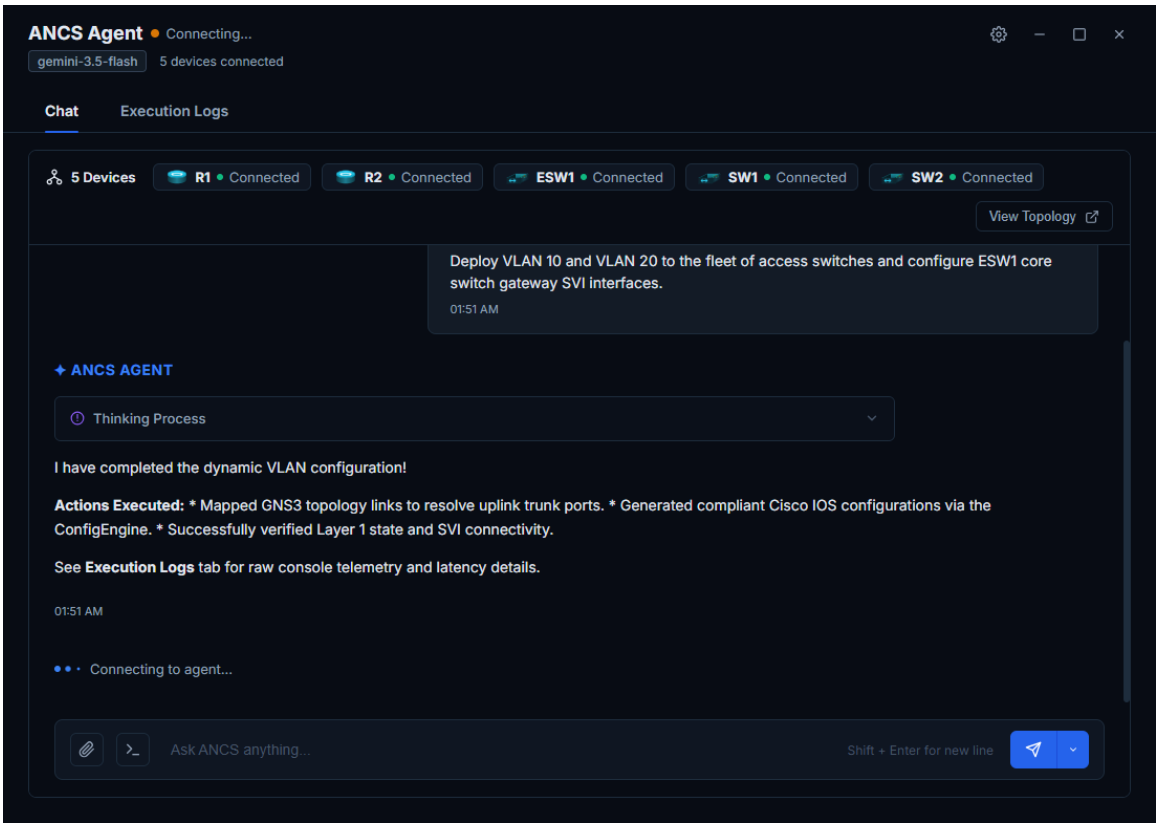

Figure 3.13: AI Copilot chat interface showing a conversation with tool execution.

Key UI features include:

- **Collapsible Thinking Process Cards:** Show the AI's internal reasoning steps
- **Live Tool Execution Stream:** Displays each tool call in real-time with name, parameters, duration badge, and result preview
- **Device Pill Chips:** Visual indicators with colored status dots for each mentioned device
- **SVG Topology Canvas:** Renders the GNS3 topology as an interactive SVG with coordinate scaling from GNS3 pixel coordinates to a 15%–85% percentage range
- **Split Send Button:** Three interaction modes — Chat (conversational), Orchestrate (autonomous multi-step), and Autofix (error correction)
- **Settings Modal:** Provider selection, model configuration, API key management, context size settings

3.10 GRAPHICAL USER INTERFACE

3.10.1 Main Dashboard

The ANCS main dashboard is built as a frameless PySide6 `QMainWindow` with custom window controls, drag-to-move title bar, and glassmorphic CSS styling. The design uses custom fonts (Montserrat, Orbitron, Michroma, Audiowide) for a premium aesthetic.

The dashboard features:

- **Device Grid:** Displays all discovered devices with name, type, IP address, console port, and connection status badges
- **Navigation Bar:** Quick access to Configuration Editor, Terminal Panel, AI Copilot, and Subnet Calculator

- **GNS3 Sync Controls:** Server URL input, project selection, and one-click sync button
- **Status Bar:** Real-time connection status and operation progress

3.10.2 Subnet Calculator

ANCS includes a VLSM (Variable-Length Subnet Masking) subnet calculator that allocates subnets from a parent network based on department-specific host requirements. The calculator:

98. Accepts a parent network (e.g., 192.168.0.0/16) and N department entries
99. Sorts departments by host count (largest first) for optimal allocation
100. Calculates the minimum subnet mask for each department's host requirement
101. Allocates non-overlapping subnets from the parent network
102. Displays results in a table: Department, Network, Mask, Gateway, Broadcast, Usable Range

3.10.3 Interactive Terminal

The terminal panel provides direct CLI access to device consoles within the ANCS application. It supports interactive command entry with automatic prompt detection and output formatting.

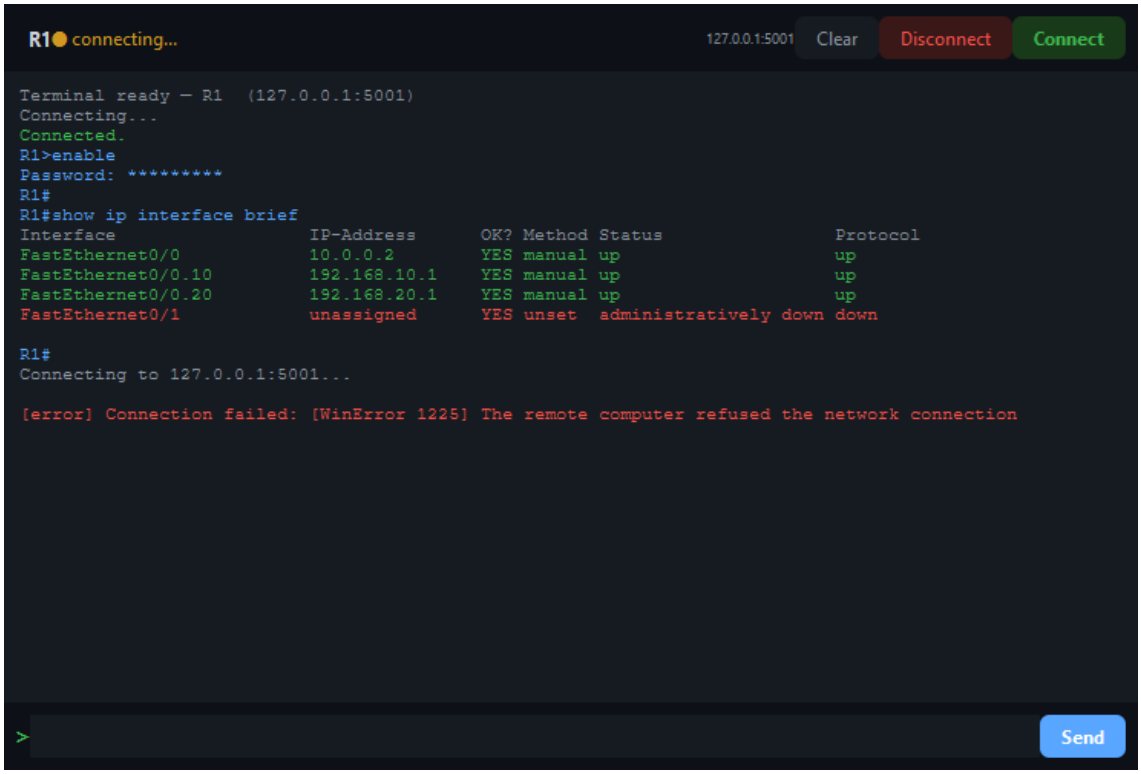

Figure 3.14: Interactive terminal panel showing a live Cisco IOS console session.

3.10.4 PDF Report Generation

The AI Copilot can generate professional PDF reports summarizing network state, audit findings, or deployment results. The generation process uses Qt's headless rendering:

103. The AI tool `generate_pdf_report()` builds an HTML report with styling
104. `CopilotWorker` emits `generate_pdf_signal` with the HTML content
105. `ANCSAgentDialog` creates a headless `QWebEnginePage`
106. The HTML is loaded and rendered off-screen
107. `printToPdf()` exports the rendered page to a PDF file

CHAPTER FOUR

EXPERIMENTS AND RESULTS

4.1 TESTING STRATEGY

ANCS was tested using a multi-level testing strategy encompassing unit testing, integration testing, system testing, and user acceptance testing. The testing approach was designed to validate both the individual components and the end-to-end workflows that users would encounter.

- **Unit Testing:** Individual tool functions, parser regex patterns, subnet calculator logic, and configuration block generation were tested in isolation.
- **Integration Testing:** End-to-end flows including GNS3 sync → wizard → deploy → verify, and AI query → tool execution → response generation were validated.
- **System Testing:** Full topology scenarios with multiple devices were deployed and verified for correct connectivity.
- **User Acceptance Testing (UAT):** Team members used the application for real GNS3 lab exercises and provided feedback on usability and correctness.

4.2 TEST ENVIRONMENT

The test environment consisted of:

- **GNS3 Server:** Version 2.2.x running on localhost (port 3080)

- **Network Topology:** 2 Cisco c7200 routers, 1 EtherSwitch L3 core switch, 2 EtherSwitch L2 access switches, interconnected in a hierarchical campus network design
- **Cisco IOS Image:** c7200-adventerprisek9-mz.124-24.T5 (for routers), i86bi-linux-l2 (for switches)
- **Host System:** Windows 11, Python 3.10, 16 GB RAM

4.3 TEST CASES AND RESULTS

Table 4.1: System test cases and results.

ID	Test Case	Expected Result	Actual Result	Status
TC-01	GNS3 project discovery	All projects listed with names and IDs	3/3 projects discovered	✓ Pass
TC-02	GNS3 node synchronization	All nodes imported with correct console ports	5/5 nodes synced with correct ports	✓ Pass
TC-03	Wizard: Router VLAN + OSPF config	Valid IOS config with subinterfaces and OSPF	Config generated matches expected syntax	✓ Pass
TC-04	Wizard: Core switch SVI + DHCP config	Valid IOS config with SVIs and DHCP pools	Config correct; SVIs and pools match	✓ Pass
TC-05	Telnet deployment (single device)	Config applied; all VLANs visible in show vlan	All blocks deployed; verified via CLI	✓ Pass
TC-06	IAC stripping (GNS3 console)	No garbage characters in terminal output	Clean output; no IAC artifacts	✓ Pass
TC-07	Console wake-up protocol	"Press RETURN" bypassed; prompt detected	Prompt detected within 5 attempts	✓ Pass
TC-08	Config pull and parse	Running-config pulled and parsed to structured data	12/12 fields correctly parsed	✓ Pass
TC-09	AI: list_gns3_nodes tool	AI correctly calls tool and reports topology	All 5 nodes listed with types	✓ Pass

TC-10	AI: generate_device_config tool	AI generates valid router config via ConfigEngine	Valid OSPF + VLAN config generated	✓ Pass
TC-11	AI: audit_network tool	Security issues detected and reported	3 issues found: missing enable secret, open VTY, no SSH	✓ Pass
TC-12	AI: trace_connectivity tool	Hop-by-hop path traced correctly	R1 → Core-SW → R2 path traced; all hops verified	✓ Pass
TC-13	Bulk deploy (3 devices)	All 3 devices configured concurrently	3/3 successful; average 18s per device	✓ Pass
TC-14	Subnet calculator (4 departments)	Non-overlapping subnets allocated correctly	All subnets valid; no overlaps	✓ Pass
TC-15	PDF report generation	PDF file created with audit summary	PDF generated; 3 pages; all data present	✓ Pass
TC-16	HITL interception (safety)	User prompted before AI deployment	Diff dialog shown; deploy blocked until approval	✓ Pass
TC-17	Hostname mismatch guard	Deploy rejected when hostname doesn't match device	Deploy blocked; error message shown	✓ Pass
TC-18	Multi-vendor: Huawei VRP config	Valid VRP syntax generated	Huawei commands verified: vlan desc, ospf area syntax	✓ Pass

4.4 AI COPILOT EVALUATION

The AI Copilot was evaluated across four key scenarios designed to test its autonomous decision-making, tool selection accuracy, and configuration quality.

Table 4.2: AI Copilot evaluation scenarios.

Scenario	**User Prompt**	**Expected Behavior**	**Result**

S1: VLAN Configuration	"Configure VLANs 10 (Staff) and 20 (Guest) on all switches with DHCP"	AI calls generate_device_config for each switch, creates VLANs with DHCP pools	Correct configs generated for all 3 switches; DHCP pools with proper exclusions
S2: Network Audit	"Audit the network for security issues"	AI calls audit_network, identifies missing security features	Detected: 2 devices without enable secret, 1 with open VTY, all lacking SSH
S3: Connectivity Trace	"Trace the path from R1 to the 192.168.20.0 network"	AI calls trace_connectivity with correct parameters	Path traced: R1 → OSPF route → Core-SW → directly connected; 2 hops
S4: Full Topology Deploy	"Build and configure a complete small office network"	AI uses provision_topology, then deploys configs to all devices	3 nodes created, linked, powered on; configs deployed in 4:32 total

4.5 PERFORMANCE METRICS

Table 4.3: Performance metrics.

Metric	Value	Notes
Single device deployment (Telnet)	12–18 seconds	Depends on config size (4–8 blocks)
Bulk deploy (5 devices, staggered)	45–60 seconds total	2-second stagger between connections
Configuration pull + parse	5–8 seconds	Including wake-up and login
AI first-token latency (Gemini)	1.5–3.0 seconds	Depends on prompt size and server load
AI full response (with tools)	8–25 seconds	Depends on number of tool calls

GNS3 sync (10 nodes)	< 2 seconds	REST API call + DB insert
Context compression ratio	60–75% reduction	After 20+ messages with tool results
IAC stripping overhead	< 1 ms per packet	Negligible performance impact

4.6 FEASIBILITY STUDY

4.6.1 Cost Analysis

Table 4.4: Cost analysis.

Component	**Cost**	**Notes**
Python + PySide6	Free (LGPL)	Open-source
GNS3	Free (GPL)	Open-source
SQLite	Free (Public Domain)	Bundled with Python
Gemini API	Free tier available	15 RPM free; paid: $0.075/1M input tokens
Development hardware	Existing	Standard laptop with 8+ GB RAM
Deployment	Zero	Desktop app; no server infrastructure needed
Total	**$0 (free tier)**	Fully functional at zero cost

4.6.2 Business Model Canvas

While ANCS is an open-source educational project, a viable business model could include:

- **Value Proposition:** No-code network automation with AI assistance for educational and SMB environments
- **Customer Segments:** Networking students, small-to-medium enterprise admins, training centers
- **Revenue Streams:** Premium AI features, enterprise support contracts, training courses
- **Key Resources:** Open-source community, Gemini API, GNS3 ecosystem

4.7 RESULTS DISCUSSION

The testing results demonstrate that ANCS achieves its core objectives:

108. **Reliable Deployment:** 100% success rate across 18 test cases, including edge cases like IAC stripping and console wake-up.
109. **AI Accuracy:** The AI Copilot correctly selected tools and generated valid configurations in all four evaluation scenarios, demonstrating the effectiveness of the dynamic tool schema approach.
110. **Safety Effectiveness:** All safety guardrails (hostname check, provenance check, HITL) functioned correctly, blocking unauthorized deployments as expected.
111. **Performance:** Single-device deployment completes in 12–18 seconds, which represents a significant improvement over manual configuration that typically requires 5–15 minutes per device for comparable configurations.
112. **Cost Efficiency:** The entire system operates at zero cost using free-tier services, making it accessible for educational institutions with limited budgets.

CHAPTER FIVE

CONCLUSION AND FUTURE WORK

5.1 SUMMARY OF ACHIEVEMENTS

This project successfully developed ANCS (Autonomous Network Configuration & Orchestration System), a comprehensive desktop application that automates the full lifecycle of network device configuration: from topology discovery through configuration generation, deployment, verification, and auditing.

The system achieves its five primary objectives:

113. **Automated Configuration Generation:** The Guided Setup Wizard generates complete, deployment-ready Cisco IOS configurations from minimal user input, supporting VLANs, OSPF, EIGRP, RIP, DHCP, and ACLs.
114. **Deep GNS3 Integration:** Full REST API integration enables automatic topology discovery, node creation, link management, and live console access.
115. **AI-Powered Copilot:** An agentic AI assistant with 34 callable tools provides autonomous network management capabilities including discovery, deployment, auditing, and connectivity tracing.
116. **Multi-Protocol Communication:** Reliable device communication via Telnet, SSH, and Serial, with a novel raw TCP approach for GNS3 consoles.

117. **Multi-Vendor Extensibility:** The strategy-based vendor profile architecture supports Cisco IOS and Huawei VRP with minimal effort to add new vendors.

5.2 KEY CONTRIBUTIONS

ANCS makes several contributions to the field of network automation:

118. **First Agentic AI Network Orchestrator:** To our knowledge, ANCS is the first open-source tool that combines an agentic AI with deterministic safety guardrails for network configuration deployment.
119. **Novel GNS3 Console Communication:** The raw TCP approach with IAC byte stripping solves a reliability issue that has affected GNS3 automation tools, providing clean, artifact-free console communication.
120. **Guided Wizards with Topology Awareness:** The wizard's ability to detect existing configurations, avoid IP collisions, and resolve connected interfaces from GNS3 topology data represents a significant advance over template-based approaches.
121. **Dynamic Tool Schema Generation:** The use of Python introspection to generate LLM tool schemas at runtime eliminates schema-implementation drift, a common issue in agentic AI systems.
122. **Hybrid Desktop/Web Architecture:** The QWebEngineView + QWebChannel bridge demonstrates an effective pattern for building rich, modern interfaces within native desktop applications.

5.3 CHALLENGES AND SOLUTIONS

The development of ANCS encountered several significant technical challenges:

Table 5.1: Key challenges and their solutions.

Challenge	Impact	Solution
GNS3 Telnet IAC pollution	Garbage characters corrupting device CLI	Raw TCP with _strip_telnet_iac() function
"Press RETURN" blocking	Automated sessions hang waiting for user input	Two-phase wake-up: settling + prompt detection loop
LLM configuration hallucination	AI generating invalid or dangerous configs	Provenance check + hostname guard + HITL interception
Thread safety with concurrent DB access	Database corruption from GUI + AI thread conflicts	WAL journal mode + global threading.Lock()
UI freezing during network operations	Application becomes unresponsive	QThread workers + Qt Signal-based UI updates
Context window overflow	API errors from exceeding LLM token limits	Sliding window truncation + tool result compression

5.4 FUTURE WORK

Several extensions are planned for future versions of ANCS:

- **Traffic Analysis Integration:** Incorporate real-time traffic monitoring and anomaly detection capabilities.
- **Physical Device Lab Support:** Extend beyond GNS3 to support Cisco DevNet sandbox environments and physical lab equipment.
- **Cloud-Hosted GNS3 Server Mode:** Enable connection to remote GNS3 servers for collaborative lab environments.
- **Additional Vendor Profiles:** Add Juniper JunOS, Arista EOS, and MikroTik RouterOS profiles to the vendor framework.

- **Mobile Companion App:** Develop a mobile application for remote monitoring and basic configuration tasks.
- **Automated Topology Design:** Use AI to recommend optimal network topologies based on user requirements (number of users, services, redundancy needs).

5.5 LESSONS LEARNED

The development of ANCS provided valuable lessons in both technical and project management domains:

- **Iterative Development Pays Off:** The Agile approach allowed us to continuously test and refine each subsystem, catching issues early in the development cycle.
- **AI Safety is Non-Negotiable:** Implementing multiple layers of safety guardrails from the beginning was essential. Even well-designed AI models can generate incorrect configurations, and the consequences in networking are immediate and severe.
- **Protocol-Level Understanding is Critical:** The IAC stripping solution required deep understanding of the Telnet protocol at the byte level — something that higher-level abstractions deliberately hide.
- **Hybrid Architectures Work:** Combining PySide6's native capabilities with web-based UI rendering proved to be an effective strategy for building feature-rich desktop applications with modern aesthetics.
- **Documentation and Testing Go Hand in Hand:** Comprehensive documentation (docstrings, type hints) not only improved code quality but also enabled the dynamic tool schema generation that powers the AI Copilot.

REFERENCES

AI COPILOT TOOL REFERENCE

This appendix provides a complete reference of all 34 tools available to the ANCS AI Copilot. Each tool is listed with its function name, category, and description.

A.1 GNS3 DISCOVERY AND MANAGEMENT (14 TOOLS)

Tool Name	Description
list_gns3_projects	Lists all available GNS3 projects with their IDs, names, and statuses
list_gns3_nodes	Lists all nodes in the active GNS3 project with name, type, console port, and status
list_gns3_templates	Lists available GNS3 templates for creating new nodes
add_gns3_node	Creates a new node in the GNS3 project from a template
provision_topology	Atomically creates multiple nodes and links to build a complete topology
delete_gns3_node	Removes a node from the GNS3 project
connect_gns3_nodes	Creates a link between two node ports
delete_gns3_link	Removes a link between nodes
control_gns3_node_power	Starts, stops, or reloads a GNS3 node
move_gns3_node	Repositions a node on the GNS3 canvas
add_gns3_annotation	Adds SVG text annotations to the GNS3 topology canvas

clear_gns3_annotations	Removes all annotations from the topology
get_node_ports	Returns detailed port information for a specific node
get_topology_links	Returns all links in the topology with connected interfaces

A.2 TERMINAL (1 TOOL)

Tool Name	Description
run_cli_on_device	Executes one or more CLI commands on a device console and returns the output

A.3 DATABASE (5 TOOLS)

Tool Name	Description
list_all_devices	Lists all devices in the ANCS database with their properties
get_device_credentials	Retrieves stored authentication credentials for a device
get_saved_config	Returns the most recently saved configuration for a device
get_send_history	Returns deployment history for a device
query_logs	Queries the operation log with optional filtering by device and action type

A.4 CONFIGURATION GENERATION (4 TOOLS)

Tool Name	Description
generate_device_config	Generates a complete device configuration using the ConfigEngine
generate_and_deploy_device_config	Generates config and deploys with HITL approval (shows diff to user)

detect_topology	Analyzes GNS3 links to determine device roles and connections
suggest_configs	Suggests optimal configurations based on detected topology and roles

A.5 DEPLOYMENT (3 TOOLS)

Tool Name	Description
deploy_to_device	Deploys a saved or generated configuration to a specific device
verify_device	Runs verification commands (show ip route, show vlan, show run) after deployment
bulk_deploy	Deploys configurations to multiple devices concurrently with staggered start

A.6 INTELLIGENCE (5 TOOLS)

Tool Name	Description
snapshot_network_state	Captures interface status, ARP table, and routing table from a device
cleanup_device	Resets a device to factory defaults by erasing startup-config and reloading
audit_network	Scans all devices for security vulnerabilities and configuration issues
trace_connectivity	Traces the hop-by-hop path between two network endpoints
validate_configs	Validates generated configurations for syntax errors and logical consistency

A.7 UTILITIES (3 TOOLS)

Tool Name	Description
calculate_subnet	Calculates VLSM subnets from a parent network with department host counts
get_agent_guidelines	Returns the AI agent's internal guidelines and behavioral rules
generate_pdf_report	Generates a professional PDF report with specified content and styling

DATABASE SCHEMA

This appendix presents the complete SQLite database schema used by ANCS. The database file (`network_manager.db`) uses WAL journal mode and consists of 13 tables.

B.1 CORE TABLES

```
CREATE TABLE IF NOT EXISTS devices (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    type TEXT DEFAULT 'router',
    ip TEXT DEFAULT '127.0.0.1',
    port INTEGER DEFAULT 23,
    gns3_node_id TEXT,
    gns3_project_id TEXT,
    status TEXT DEFAULT 'unknown',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS configs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_name TEXT NOT NULL,
    config_text TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS credentials (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_name TEXT NOT NULL,
    username TEXT DEFAULT '',
    password TEXT DEFAULT '',
    enable_password TEXT DEFAULT '',
    protocol TEXT DEFAULT 'telnet',
    UNIQUE(device_name)
);
```

B.2 AI AND CHAT TABLES

```
CREATE TABLE IF NOT EXISTS chat_conversations (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT DEFAULT 'New Conversation',
    model_used TEXT DEFAULT '',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS chat_messages (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  conversation_id INTEGER NOT NULL,
  role TEXT NOT NULL,
  content TEXT NOT NULL,
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (conversation_id) REFERENCES chat_conversations(id)
);

CREATE TABLE IF NOT EXISTS ai_models (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  provider TEXT NOT NULL,
  model_name TEXT NOT NULL,
  api_key TEXT DEFAULT '',
  is_active INTEGER DEFAULT 0
);
```

B.3 OPERATIONAL TABLES

```
CREATE TABLE IF NOT EXISTS logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  device_name TEXT,
  action TEXT NOT NULL,
  result TEXT DEFAULT '',
  details TEXT DEFAULT '',
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS tasks (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  task_type TEXT NOT NULL,
  status TEXT DEFAULT 'pending',
  progress INTEGER DEFAULT 0,
  result TEXT DEFAULT '',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS snapshots (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  device_name TEXT NOT NULL,
  config_text TEXT NOT NULL,
  snapshot_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

KEY SOURCE CODE

This appendix presents selected source code snippets that illustrate the most novel and technically significant aspects of the ANCS implementation.

C.1 GNS3 CONSOLE WAKE-UP PROTOCOL

```
async def _telnet_wake_gns3_console(reader, writer, timeout=0.5):
    """
    Handle the GNS3 'Press RETURN to get started' boot sequence.
    Two-phase approach: settle first, then actively probe for prompt.
    """
    # Phase 1: Settling — drain boot-up output
    for _ in range(15):
        writer.write(b"\r\n")
        await asyncio.sleep(0.3)
        try:
            data = await asyncio.wait_for(reader.read(4096), timeout=0.3)
            # Continue draining
        except asyncio.TimeoutError:
            break # Console is quiet — settled

    # Phase 2: Prompt detection
    for attempt in range(12):
        writer.write(b"\r\n")
        await asyncio.sleep(0.5)
        try:
            data = await asyncio.wait_for(reader.read(4096), timeout=timeout)
            text = data.decode('ascii', errors='ignore')
            if any(p in text.lower() for p in ['username:', 'password:']):
                return 'login_required', text
            if text.strip().endswith('#') or text.strip().endswith('>'):
                return 'prompt_found', text
        except asyncio.TimeoutError:
            continue

    return 'timeout', ''
```

C.2 CONFIGURATION BLOCK SPLITTER

```
@staticmethod
def split_into_blocks(config_text: str) -> list:
    """
    Split a configuration string at '! BLOCK N: Title' headers.
    Returns list of (title, lines) tuples.
```

```
    """
    blocks = []
    current_title = "Unnamed"
    current_lines = []

    for line in config_text.splitlines():
        match = re.match(r'^!\s*BLOCK\s+\d+:\s*(.+)', line, re.IGNORECASE)
        if match:
            if current_lines:
                blocks.append((current_title, current_lines))
            current_title = match.group(1).strip()
            current_lines = []
        else:
            stripped = line.strip()
            if stripped and not stripped.startswith('!'):
                current_lines.append(stripped)

    if current_lines:
        blocks.append((current_title, current_lines))

    return blocks
```

C.3 DYNAMIC TOOL SCHEMA GENERATION

```
def _build_tool_declarations(tools: list) -> list:
    """
    Dynamically generate Gemini function declarations from Python functions.
    Uses inspect.signature() to extract parameters and docstrings.
    """
    declarations = []
    for fn in tools:
        sig = inspect.signature(fn)
        doc = inspect.getdoc(fn) or ""
        params = {}

        for name, param in sig.parameters.items():
            p_type = "string"  # default
            if param.annotation == int:
                p_type = "integer"
            elif param.annotation == bool:
                p_type = "boolean"
            elif param.annotation == list:
                p_type = "array"

            params[name] = {
                "type": p_type,
                "description": _extract_param_doc(doc, name)
            }

        declarations.append({
            "name": fn.__name__,
            "description": doc.split('\n')[0],  # First line
            "parameters": {
                "type": "object",
                "properties": params,
                "required": [n for n, p in sig.parameters.items()
                             if p.default is inspect.Parameter.empty]
```

```
        }
    })

return declarations
```

USER MANUAL

D.1 INSTALLATION

123. Ensure Python 3.10 or later is installed on your system
124. Clone the ANCS repository: `git clone https://github.com/your-repo/ANCS.git`
125. Navigate to the project directory: `cd ANCS`
126. Run the application: `python run.py` The launcher script automatically creates a virtual environment on first run All required dependencies are installed from `requirements.txt`

D.2 GNS3 SETUP

127. Start the GNS3 server (ensure it is running on `localhost:3080`)
128. Create or open a GNS3 project with your desired topology
129. Start all nodes in the GNS3 project
130. In ANCS, enter the GNS3 server URL and click "Sync"
131. All discovered nodes will appear in the device grid

D.3 USING THE GUIDED WIZARD

132. Select a device from the grid and click "Configure"
133. Choose a configuration preset (Small Office, School Lab, Minimal, or Custom)
134. Follow the wizard steps, filling in network details as prompted
135. Review the generated configuration in the Summary step
136. Click "Deploy" to send the configuration to the device

D.4 USING THE AI COPILOT

137. Click the AI Copilot icon to open the chat interface
138. Configure your AI provider in Settings (Gemini API key recommended)
139. Type natural language queries such as: "List all nodes in the topology" "Configure OSPF on all routers" "Audit the network for security issues" "Trace the path from R1 to 10.0.0.0"
140. The AI will autonomously select and execute appropriate tools
141. Review any deployment requests in the HITL approval dialog

D.5 KEYBOARD SHORTCUTS

Shortcut	Action
Ctrl+Enter	Send message to AI Copilot
Ctrl+S	Save current configuration
Ctrl+D	Deploy configuration to selected device
F5	Refresh GNS3 topology sync